第二章 数据的表示与运算

SEU.CSE.RGL

※主要内容

(1)数据的编码

机器数编码、十进制数编码(△)、字符编码，校验码(×)

(2)数据的表示

数据表示方法，整数的表示(含位扩展运算实现)，实数的表示，非数值数据的表示(含逻辑运算/关系运算实现方法)

(3)数据的运算

定点运算—有/无符号的加减、移位、乘法(△)运算(方法/部件组织)

浮点运算—加减(△)运算(方法)

(4)运算器的组成

加法器的组成，运算器的组成(功能组织/部件设计/部件连接)

※总体要求：掌握数据表示方法，可组织运算方法及运算部件

§2.1 数据的编码

※主要内容：进制转换，机器数/十进制数/字符的编码，校验码

一、数制及其转换

1、进位计数制　　--又称进制或数制

*常用的4种进制：参数有数码、基数、位权(进位规则)

	二进制	八进制	十进制	十六进制
数码	0, 1	0, 1, …, 7	0, 1, …, 9	0, 1, …, 9, A, B, …, F
基数	2	8	10	16
位权	2^i	8^i	10^i	16^i
书写(后缀)	B	O	D(缺省)	H

注：B—Binary，O—Octal，D—Decimal，H—Hexadecimal

*R进制数的表示：$(N)_R=(k_{n-1}\cdots k_1k_0.k_{-1}k_{-2}\cdots k_{-m})_R=\sum_{i=-m}^{n-1}k_i\times R^i$

其中，$k_i\in\{0,1,\cdots,R-1\}$

思考：如何记A~F对应的值？C语言是如何表示16进制数的？

2、不同进制数的转换

*R进制数→十进制数的转换：按位权展开

例1—$(101.01)_2=(1\times2^2+1\times2^0+1\times2^{-2})_{10}=5.25D$

$3A.CH=(3\times16^1+10\times16^0+12\times16^{-1})_{10}=58.75$

*十进制数→R进制数的转换：整数与小数分别转换，

整数为除基取余/减权定位法，小数为乘基取整/减权定位法

例2—$19.625=(2^4+2^1+2^0)+(2^{-1}+2^{-3})=10011.101B$

2 | 19	1	(最低位)
2 | 9	1	
2 | 4	0	
2 | 2	0	
2 | 1	1	(最高位)
0	余数	

$19-2^5 \quad <0$、

$19-2^4=3\geqslant0,\ k_4=1$

$3-2^3 \quad <0,\ k_3=0$

$3-2^2 \quad <0,\ k_2=0$

$3-2^1=1\geqslant0,\ k_1=1$

$1-2^0=0\geqslant0,\ k_0=1$

$0.625\times2=1.25$ 1(最高位)

$0.25\times2=0.5$ 0 ↓

$0.5\times2=1.0$ 1(最低位)

$0.625-2^{-1}=0.125\geqslant0,\ k_{-1}=1$

$0.125-2^{-2} \quad <0,\ k_{-2}=0$

$0.125-2^{-3}=0 \quad =0,\ k_{-3}=1$

*二-八/十六进制数的转换：

整数与小数分别转换，数位不够时向两边补零(长度1:3/4)

练习—$(97.75)_{10}=(\quad)_2=(\quad)_8=(\quad)_{16}$，

$2D.EH=\quad B=\quad O=\quad D$

二、机器数及其编码

*数值数据：

组成— [符号] 数码 [小数点 数码]　　←[ ]表示可缺省

如：$(10)_2$、$(+10)_2$、$(-10)_2$、$(-10.1)_2$

运算特征— 符号与数值分开运算，减法先比较大小

*机器数：指计算机内部用编码表示的数据

└→二进制→┘　　└←含符号、小数点

如：$(+10)_2 \rightarrow (010)_2$、$(-10)_2 \rightarrow (110)_2$、$(-10.1)_2 \rightarrow (110101)_2$

真值—指机器数真正的值，即数学上的数据

*机器数的编码：

目标— 便于硬件实现运算(如符号/数值一起运算、减法不比较大小)

类型— 原码、补码、移码等，均用二进制表示

PPT约定：本章数据的书写方式默认为二进制、省略后缀B，特殊说明除外

1、原码(sign-magnitude)表示法

*编码思想：机器数的最高位(符号位)用0/1表示+/−，
其余位(数值位)为真值的绝对值

*整数原码定义：$X=\pm x_{n-2}\cdots x_0$，$x_i=0$或1，$[X]_{原}=x_{n-1}x_{n-2}\cdots x_0$

例1—$[+110]_{原}=0110$，$[-110]_{原}=1110$；$[X]_{原}=1101$时$X=-101$

$[+000]_{原}=0000$，$[-000]_{原}=1000$　※$[+0]_{原}\neq[-0]_{原}$

*小数原码定义：$X=\pm 0.x_{-1}\cdots x_{-(n-1)}$，$[X]_{原}=x_0.x_{-1}\cdots x_{-(n-1)}$

└——→编码时省略0→——┘　└←硬件隐含表示(此处便于阅读)

例2—$[+0.101]_{原}=0.101$；$[-0.101]_{原}=1.101$

若$[X]_{原}=1.01$，则$X=-0.01$

*原码的特性：

表示特性—$[X]_{原}$与X表示值的范围相同　←$[+0]_{原}\neq[-0]$

运算方法—符号与数值分开运算、减法先比较大小

└→不利于硬件实现←┴←常用减法实现

2、补码(two's complement)表示法

编码目标—①符号与数值一起运算，②减法不比较大小

*有模运算：仅计量小于"模"的部分，其余部分被丢弃

└←计量系统的计数范围，适于定长运算

如：时针(模＝12)从10点拨到7点，10－3＝10＋9＝7 (mod 12)

同余—若A、B、M满足$A=B+kM$ (k为整数)，

则记 $A\equiv B \pmod{M}$，称B和A为模M的同余

如：$+3\equiv+15 \pmod{12}$，$-3\equiv+9 \pmod{12}$

补数—若a、b、M满足$a+b=M$，称a、b互为模M的补数

运算特征：$c-a=c-(M-b)=c+b \pmod{M}$

└→减法可用加法实现←┴→可简化硬件

*编码目标的实现：　(硬件是有模运算)

目标②—正数用其本身、负数用其正补数表示 ←减法用加法实现

如模12时，$9-4\equiv9+8$，$4+(-9)\equiv4+3$

目标①—结果符号等于其值类型(本身/正补数)　←下节证明

(1)补码定义

*编码思想：机器数的最高位(符号位)用0/1表示+/-，
其余位(数值位)为本身或正补数

*整数补码定义：$X=\pm x_{n-2}\cdots x_0$，则模为2^n，$[X]_{补}=x'_{n-1}x'_{n-2}\cdots x'_0$

$$[X]_{补}=2^n+X \pmod{2^n}=\begin{cases}0\times 2^{n-1}+X & 0\leqslant X<2^{n-1}\\ 1\times 2^{n-1}+(2^{n-1}-|X|) & -2^{n-1}\leqslant X<0\end{cases}$$

＝正补数

例3—$[+001]_{补}=0001$，…，$[+111]_{补}=0111$

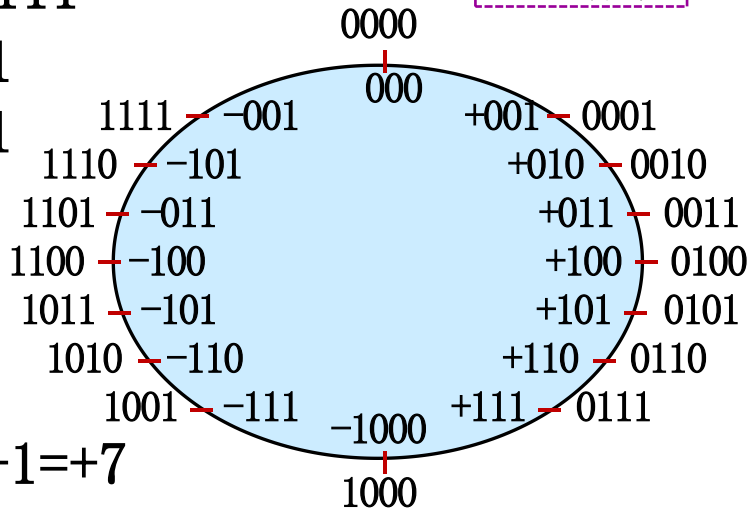
$[-001]_{补}=1000+(1000-001)=1111$

$[-111]_{补}=1000+(1000-111)=1001$

练习1—若A=-01100、求$[A]_{补}$

例4—$[+000]_{补}=0000$，$[-000]_{补}=0000$

※数0的补码唯一



例5—X≥0时，$[X]_{补}\leqslant 0111$，$X_{max}=2^{4-1}-1=+7$

X<0时，$[X]_{补}\geqslant 1000$，$X_{min}=-2^{4-1}=-8$

※补码比原码多表示1个数

*小数补码定义：$X=\pm 0.x_{-1}\cdots x_{-(n-1)}$，则$[X]_{补}=x'_0.x'_{-1}\cdots x'_{-(n-1)}$

(2) 补码的特性

+1不能影响符号位($x_{n-2}\cdots x_0=0\cdots0$除外)

***X与$[X]_{补}$的关系：** 设$X=\pm x_{n-2}\ldots x_0$，$[Y]_{补}=y_{n-1}y_{n-2}\cdots y_0$

①$X\geqslant0$时，$[X]_{补}=0\,x_{n-2}\cdots x_0$；$y_{n-1}=0$时，$Y=+y_{n-2}\cdots y_0$

②$X<0$时，$[X]_{补}=2^n+X=[2^{n-1}+(2^{n-1}-1)+1]+X$

$=10\cdots0+(1\cdots1-x_{n-2}\cdots x_0)+1=1\,\overline{x_{n-2}}\cdots\overline{x_0}+1$

$y_{n-1}=1$时，$Y=[Y]_{补}-2^n=1y_{n-2}\cdots y_0-[2^{n-1}+(2^{n-1}-1)+1]$

$=2^{n-1}-2^{n-1}-[(1\cdots1-y_{n-2}\cdots y_0)+1]=-\overline{y_{n-2}}\cdots\overline{y_0}+1$

$X\rightarrow[X]_{补}$简便方法—

$X\geqslant0$时$[X]_{补}=0\,x_{n-2}\cdots x_0$；

$-2^{n-1}<X<0$时$[X]_{补}=1\,\overline{x_{n-2}}\cdots\overline{x_0}+1$；

$X=-2^{n-1}$时$[X]_{补}=1\,0\cdots0$

例6—若X=+101，Y=-101，则$[X]_{补}=0101$，$[Y]_{补}=1011$

$[Y]_{补}\rightarrow Y$简便方法— $y_{n-1}=0$时$Y=+y_{n-2}\cdots y_0$；

$y_{n-1}=1$且$y_{n-2}\cdots y_0\neq0$时$Y=-\overline{y_{n-2}}\cdots\overline{y_0}+1$

$y_{n-1}=1$且$y_{n-2}\cdots y_0=0$时$Y=-2^{n-1}$

例7—若$[X]_{补}=0101$，$[Y]_{补}=1011$，则X=+101，Y=-101

*$[X]_{补}$与$[-X]_{补}$的关系：设$[X]_{补}=x_{n-1}x_{n-2}\cdots x_0$ ←减法X-Y=X+(-Y)

$X\geqslant 0$时，$x_{n-1}=0$，$[X]_{补}=X$，

$[-X]_{补}=1\overline{x_{n-2}}\cdots\overline{x_0}+1=\overline{x_{n-1}}\ \overline{x_{n-2}}\cdots\overline{x_0}+1$

$X<0$时，$x_{n-1}=1$，$[X]_{补}=2^n-|X|$，

$[-X]_{补}=|X|=2^n-[X]_{补}=1\cdots1-x_{n-1}x_{n-2}\cdots x_0+1$

$=\overline{x_{n-1}}\ \overline{x_{n-2}}\cdots\overline{x_0}+1$

+1不能影响符号位
($x_{n-2}\cdots x_0$=0…0除外)

$[X]_{补}\rightarrow[-X]_{补}$简便方法：

$[X]_{补}\neq 10\cdots0$时$[-X]_{补}=\overline{x_{n-1}}\ \overline{x_{n-2}}\cdots\overline{x_0}+1$；

$[X]_{补}=10\cdots0$时$[-X]_{补}$无法表示

例8—$[X]_{补}=0101$，$[Y]_{补}=1011$，则$[-X]_{补}=1011$，$[-Y]_{补}=0101$

注意：上述方法也适用于小数补码，

不适用于最小负数(正数/负数的表示范围不同)

练习3—

①若X=+01001，$[X]_{原}$、$[X]_{补}$分别为

A. 0 01001和0 10111　B. 1 01001和1 01001　C. 0 01001和0 01001

②若Y=−01010，$[Y]_{原}$、$[Y]_{补}$分别为

A. 1 01010和1 01010　B. 1 01010和1 10110　C. 0 01010和0 10110

③若$[X]_{原}$=001010，X、$[X]_{补}$分别为

A. +01010和1 10110　B. +10110和0 10110　C. +0 1010和0 01010

④若$[Y]_{原}$=101110，Y、$[Y]_{补}$分别为

A. −01110和1 10010　B. −10010和1 10010　C. −01110和0 10010

⑤若$[X]_{补}$=001110，X=＿＿＿＿，$[-X]_{补}$=＿＿＿＿

⑥若$[Y]_{补}$=101110，Y=＿＿＿＿，$[-Y]_{补}$=＿＿＿＿

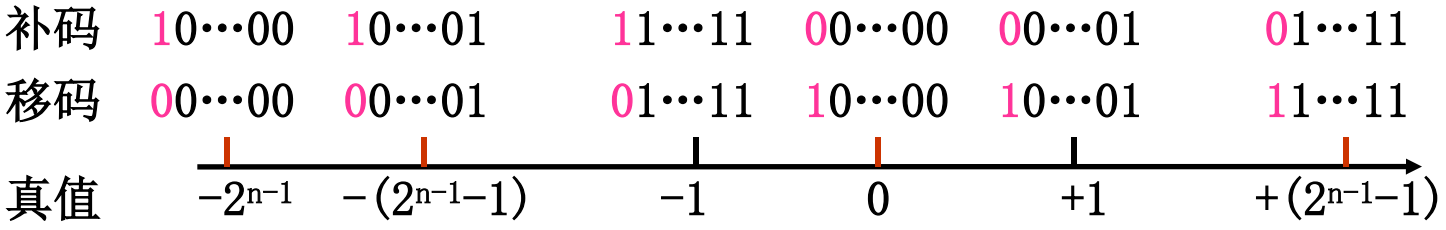
原码、补码编码规则：（设$X=\pm x_{n-2}\cdots x_0$）

$X\geqslant 0$时，$[X]_{原}=0x_{n-2}\cdots x_0$；$[X]_{补}=0x_{n-2}\cdots x_0$，$[-X]_{补}=1\overline{x_{n-2}}\cdots\overline{x_0}+1$

$X<0$时，$[X]_{原}=1x_{n-2}\cdots x_0$；$[X]_{补}=1\overline{x_{n-2}}\cdots\overline{x_0}+1$，$[-X]_{补}=0x_{n-2}\cdots x_0$

3、移码表示法

编码目标—真值连续时，机器数也连续(机器数＝真值＋偏移量)



*整数移码定义：设$X=\pm x_{n-2}\cdots x_0$，则模＝2^n、偏移量＝2^{n-1}，

$$[X]_{移}=2^{n-1}+X \pmod{2^n}=2^{n-1}+X \qquad -2^{n-1}\leqslant X<2^{n-1}$$

补码为2^n，即负数的符号位=1

例13—$[-111]_{移}=0001$，$[-001]_{移}=0111$，$[\pm 000]_{移}=1000$，

$[+001]_{移}=1001$，$[+111]_{移}=1111$，$[-1000]_{移}=0000$

*移码的特性：

①数轴上数的移码为连续编码，便于比较大小

②$[X]_{移符号位}=\overline{[X]_{补符号位}}$，$[X]_{移数值位}=[X]_{补数值位}$

※机器数编码小结：

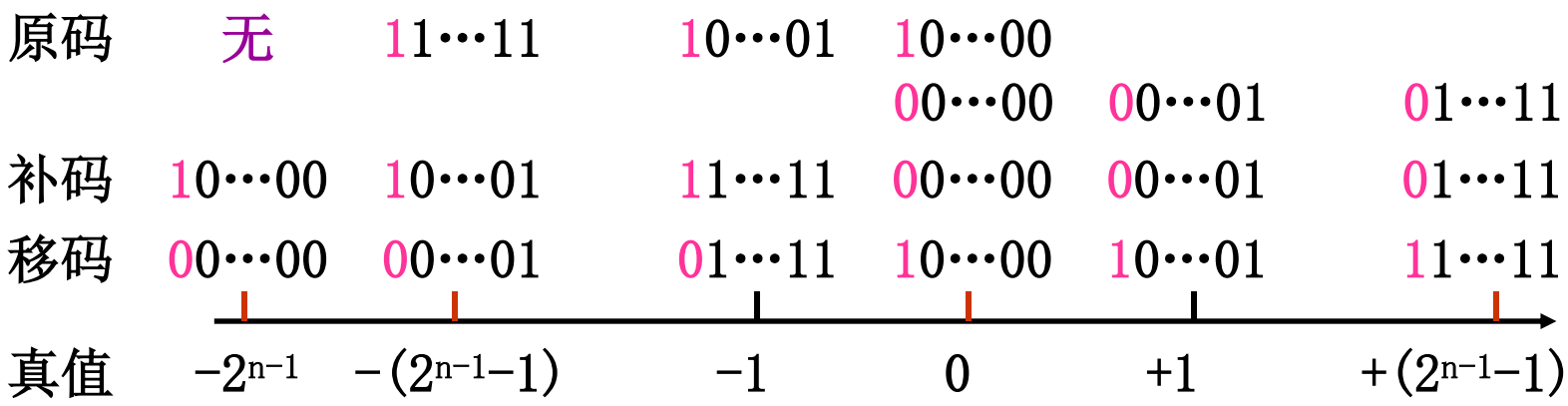
①原码/补码/移码的最高位均为符号位、其余位均为数值位

②符号位中，原码/补码用0/1表示正/负，移码反之

③数值位中，$X \geqslant 0$时，原码/补码/移码$=|X|$；

$X<0$时，原码$=|X|$，补码/移码$=\overline{|X|}+1$

④补码、移码比原码多表示1个负数



测试—X=+01001时，$[X]_{原}$=＿＿＿＿，$[X]_{补}$=＿＿＿＿，$[X]_{移}$=＿＿＿＿

Y=−01010时，$[Y]_{原}$=＿＿＿＿，$[Y]_{补}$=＿＿＿＿，$[Y]_{移}$=＿＿＿＿

三、十进制数编码

*BCD码：指用4位二进制编码表示1位十进制数位的编码方式

└←Binary Coded Decimal，二-十进制编码

*BCD编码的种类：分为有权码和无权码(格雷码等)

十进制数	0	1	2	3	4	5	6	7	8	9
8421码	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001
余3码	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100

BCD码缺省类型—8421码！　　(特殊声明除外)

*十进制数的表示：　(常仅整数)

格式—数值位＋符号位，长度＝k×存储单元长度



编码—数值位为BCD码，符号位为特殊编码

└←如+/-/无为1100/1101/1111

例— +427为0100 0010 0111 1100，-23为0000 0010 0011 1101，

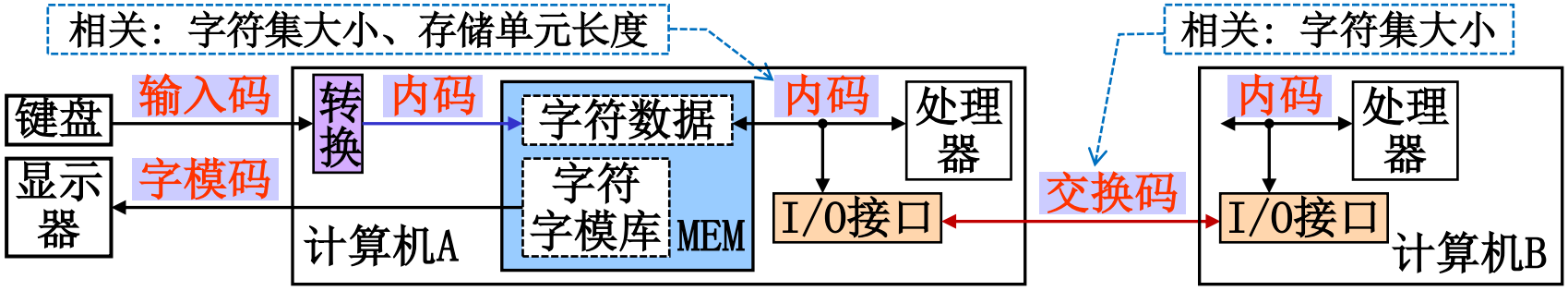
426为0100 0010 0110 1111

四、字符编码

指字符在字符集中的唯一数字化代码(表示序号或特征号)

*编码种类：交换码、内码、输入码、字模码等

- 存储时的编码(数据)，如char型字符的值
- 传送时的编码(序号)，如字符的ASCII码



*常见的字符编码：　(默认指交换码)　←内码常称为字符数据

编码类型	码点个数	编码长度	说明
ASCII码	128	7	美国标准信息交换码，英文，使用最广泛
EBCDIC码	256	8	扩展二-十进制交换码，英文，IBM定义
Unicode码	65536 2^{32}	16 32	统一字符码，有UCS-2、UCS-4两种字符集，支持各国语言，使用较广泛
ANSI码	256	8	美国国家标准协会交换码，英文，含ASCII码
GB2312-80	7445	14	汉字国标码，中文

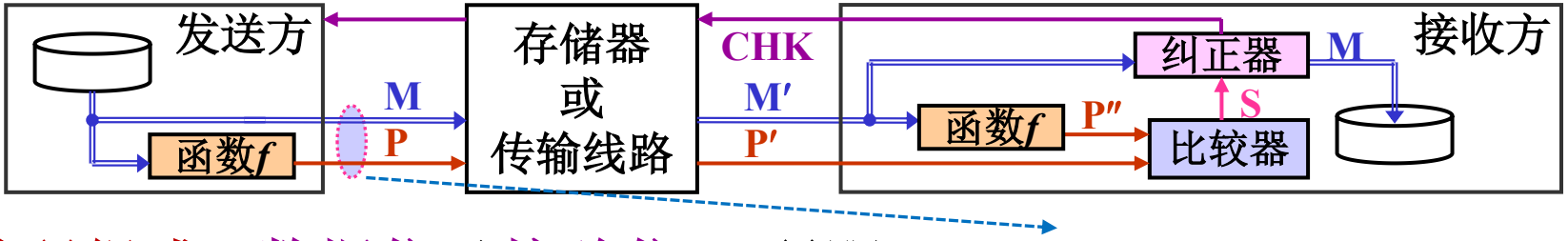
作业2-1：P86起2～6

五、校验码

*数据校验过程：　　--冗余校验思想

①用待发数据M 形成校验信息P，M与P一起传送

②用所收数据M′形成校验信息P″，比较P′及P″，校验并返回状态



*校验码组成：数据位及校验位，码距≥2(合法码字<全部编码)

码距—任意2个编码中位值不同的最小个数，如{000,011,101,110}=2

*校验原理：检错—判断所收校验码∈合法码字?

纠错—基于非法码字→定位错误→纠正错误(取反)

*检验方式：并行校验(接收完再校验)、串行校验(边接收边校验)

*校验码类型：奇偶校验码、海明校验码，循环冗余校验码(CRC)

1、奇偶校验码　（掌握）

*校验原理：编码时，使校验码中的1为奇数/偶数个；

检测校验码中的1为奇数/偶数个，验证有无错误

└→2种方法→┘

*校验码组成：



，校验位为1位(区分奇/偶)

校验位编码— 奇校验：$P=p_1=m_n\oplus\cdots\oplus m_1\oplus 1=m_n+\cdots+m_1+1 \pmod 2$

偶校验：$P=p_1=m_n\oplus\cdots\oplus m_1=m_n+\cdots+m_1 \pmod 2$

例1—10110的奇校验码为101100、偶校验码为101101

*校验方法：故障字—$S=P'\oplus P''$，P'是接收的、P''是M'形成的

检　错—$S=0$时无错误，$S=1$时有错误

可检测奇数个错←┴→无纠错能力

例2—所收校验码0101、1010、1011中有一个有错，编码方式？

*校验方式：可串行校验(逐位异或求P'')

*应用：广泛用于I/O传输方面(有甚于无)

2、海明校验码　　（了解）

*校验原理：数据分成多个有重叠的组，各组采用奇偶校验编码；
根据多重奇偶校验结果，定位(纠正)错误
└→某位错导致多个校验位错→┘

校验方法—　　（多重奇偶校验）

设数据$M=m_n\cdots m_1$，校验位$P=p_k\cdots p_1$(即有k个奇偶检验组)

故障字：$S=s_k\cdots s_1$，$s_i=p_i'\oplus p_i''=p_i'+p_i''\pmod 2$

检错/纠错：S＝0时无错误，S≠0时值表示错误位置

校验位数— k＝分组数，取决于校验能力(如纠正1位错误)

例：单纠错码SEC需满足$2^k-1\geq n+k$，

n	1	2～4	5～11	12～26	27～57
k(最小值)	2	3	4	5	6

双纠错码需满足$2^k-1\geq (n+k)+(n+k)*(n+k-1)$

*校验码的组成：　（以4个校验组的SEC码为例[M=m_{11}…m_1、P=p_4…p_1]）

错误位置的约定—用S值表示

无 错 误：0000

校验位错：0001(p_1)、0010(p_2)、0100(p_3)、1000(p_4)

数据位错：其余值(≥2个s_i=1)

→信息位置从1开始编号

←数据更重要(加入组≥2个)

校验码字的组成— 按“错误位置＝S值”规则排列

位置	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1
内容	m_{11}	m_{10}	m_9	m_8	m_7	m_6	m_5	p_4	m_4	m_3	m_2	p_3	m_1	p_2	p_1

数据的分组—有k个奇偶校验组

位置为H(=$h_k\cdots h_1$)的数据位，加入$\{i \mid h_i=1\}$的校验组(≥2个)

└→有错误时→┴→$s_i=1$→故障字S=H

位置H	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1
	1111	1110	1101	1100	1011	1010	1001	1000	0111	0110	0101	0100	0011	0010	0001
信息	m_{11}	m_{10}	m_9	m_8	m_7	m_6	m_5	p_4	m_4	m_3	m_2	p_3	m_1	p_2	p_1
第4组	√	√	√	√	√	√	√								
第3组	√	√	√	√					√	√	√				
第2组	√	√			√	√			√	√			√		
第1组	√		√		√		√		√		√		√		
故障字S	1111	1110	1101	1100	1011	1010	1001	1000	0111	0110	0101	0100	0011	0010	0001

检验位的编码—　(缺省为偶校验方式)

$$p_4=m_{11}+m_{10}+m_9+m_8+m_7+m_6+m_5 \pmod 2$$

$$p_3=m_{11}+m_{10}+m_9+m_8+m_4+m_3+m_2 \pmod 2$$

$$p_2=m_{11}+m_{10}+m_7+m_6+m_4+m_3+m_1 \pmod 2$$

$$p_1=m_{11}+m_9+m_7+m_5+m_4+m_2+m_1 \pmod 2$$

例4—若数据有16位，则海明校验码的校验位最少为多少位?

解：$2^k-1\geqslant 16+k$，k最小为5位($2^4-1<20$、$2^5-1>21$)

例5—求字符b的ASCII码($m_7\cdots m_1$=1100010)的海明偶校验码

解：∵$2^3-1<7+3$、$2^4-1>7+4$　∴校验位位数＝4位；

按故障字约定，校验码为$m_7m_6m_5p_4m_4m_3m_2p_3m_1p_2p_1$

根据数据分组规则，得　（偶校验方式）

$p_4=m_7\oplus m_6\oplus m_5=0$

$p_3=m_4\oplus m_3\oplus m_2=1$

$p_2=m_7\oplus m_6\oplus m_4\oplus m_3\oplus m_1=0$

$p_1=m_7\oplus m_5\oplus m_4\oplus m_2\oplus m_1=0$

偶校验码＝$m_7m_6m_5p_4m_4m_3m_2p_3m_1p_2p_1$＝1 1 0 0 0 0 1 1 0 0 0

例6—某海明偶校验码$m_7m_6m_5p_4m_4m_3m_2p_3m_1p_2p_1$＝11000011000，接收的校验码为①11000011010 ②11000001000 ③11001001000，写出数据校验（检错并纠错）结果。

$p_4＝m_7⊕m_6⊕m_5$
$p_3＝m_4⊕m_3⊕m_2$
$p_2＝m_7⊕m_6⊕m_4⊕m_3⊕m_1$
$p_1＝ m_7⊕m_5⊕m_4⊕m_2⊕m_1$

解：①接收的M′＝1100010、P′＝0110，

可得P″＝0100，

S＝P′＋P″(mod 2)，得S＝0010，

∴位置2(p_2)有错误，数据M＝1100010，实际如此

②M′＝1100000、P′＝0100，可得P″＝0001，则S＝0101，

∴位置5(m_2)有错误，数据M＝1100010，实际如此

③M′＝1101000、P′＝0100，可得P″＝0110，则S＝0010，

∴位置2(p_2)有错误？实际为位置7(m_4)及位置5(m_2)错

错判原因：SEC只能纠正1位错，或检测2位错！

*校验方式： 需并行校验(接收完才求P″、用译码器纠错)

*应用： 常用于并行传输、存取方面(出错概率小、可纠错)

3、循环冗余校验(Cyclic Redundancy Check, CRC)码　　(不考)

编码与多项式—$m_{n-1}\cdots m_1m_0$可用$m_{n-1}X^{n-1}+\cdots+m_1X+m_0=M(X)$表示
模2除法—根据余数首位上商，用模2减法求新余数，商无意义
生成多项式—CRC码中用作除数的多项式，常用G(X)表示

***校验原理：** 编码时使CRC码能被G(X)整除　←模2除法(只关心余数)
用CRC码除以G(X)后的余数进行校验

***校验码的组成：**

数据$m_n\cdots m_1$	校验位$r_k\cdots r_1$

校验位的编码—$r_k\cdots r_1=R(X)=(m_n\cdots m_1<<k)/G(X)$的余数

即 $M(X)\cdot X^k/G(X)=Q(X)\cdots\cdots R(X)$　(模2除)

能整除推导：
$[M(X)\cdot X^k+R(X)]/G(X)$　(模2除)
$=\{Q(X)G(X)+R(X)+R(X)\}/G(X)$　(模2除)
$=[Q(X)G(X)]/G(X)=Q(X)\cdots\cdots 0$　(模2除)

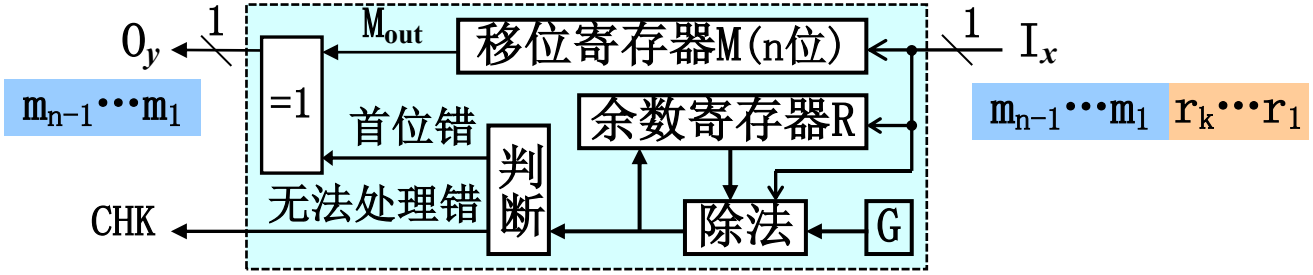
模2加法

例7—已知M(X)＝1100，G(X)＝X^3+X+1，求其CRC码

解：G(X)＝1011，1100000/1011＝1110…010　(模2除)
CRC码＝1100000＋010＝1100 010

*校验方式：可串行校验(除法是逐位相除的)

实现—



	初值	前n拍	中k拍	后n-k拍
M	0	M←M<<1+I_x		M←M<<1
R	0	R←(R<<1+I_x)/G(X)的余数		R←(R<<1)/G(X)的余数
Q_y		无效数据	校验后数据(首位错时纠错)	

对R(X)的要求—

①不同位置出错的R(X)不同 ←便于定位错误位置

②$R_{i+1}(X)=[R_i(X)<<1]/G(X)$的余数 ←便于确定首位错误(错误位置移动)

| └←称为循环码的原因

└←第i+1位出错时的余数

*G(X)的选择：

基本要求— （满足对R(X)的要求）

①任一位错误时，有R(X)不为0

②不同位错误时，有R(X)各不相同

③任一位错误时，有[$R_i(X)<<1$]/G(X)的余数＝$R_{i+1}(X)$

常用的G(X)—

CRC-CCITT：$G(X)=X^{16}+X^{12}+X^5+1$

CRC-16：$G(X)=X^{16}+X^{15}+X^2+1$

CRC-12：$G(X)=X^{12}+X^{11}+X^3+X^2+X+1$

CRC-32：$G(X)=X^{32}+X^{26}+X^{23}+X^{16}+X^{12}+X^{11}+X^{10}+X^8+X^7+X^5+X^4+X^2+X+1$

*校验能力：由G(X)决定，检错能力较强、纠错能力较弱

*应用：广泛用于网络通信等方面(检错要求高)

└→CRC＋错时重发

§2.2 数据的表示

※主要内容：数据的表示方法，整数、实数、非数值数据的表示，位扩展运算、逻辑运算、关系运算的实现

*数据类型：指一个值的集合，及定义在该值集上的一组操作

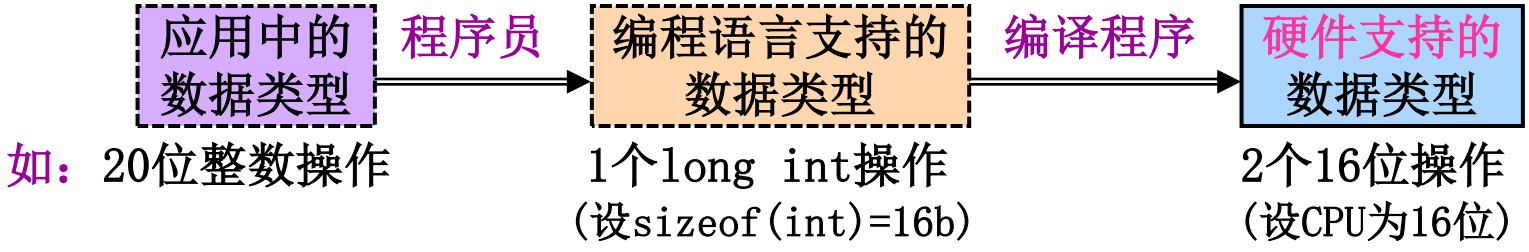
└→值集或操作集不同，数据类型不同←┘

*应用中的数据类型：

- 数据
  - 数值数据
    - 无符号数 --自然数
    - 有符号数 --整数、纯小数、实数等
  - 非数值数据
    - 逻辑数 --布尔数
    - 字符(串) --含汉字
    - 其它 --声音、图形等

*硬件支持的数据类型：最常用(最基本)的数据类型 ←性/价高

*数据类型的转换：由程序员、编译程序共同完成



一、数据的表示方法

*数据的数学特征：

数值数据—含进制&符号&小数点&编码&长度，有多种组合

非数值数据—编码非数值&长度固定，无进制&符号&小数点

*硬件的表示特征：二进制，无符号&小数点，长度固定(定长运算)

*数据的表示方法：(面向存储及运算)

进制—用二进制表示，可支持二-十进制(适于I/O较多的数据)

符号—编码中用数字表示，无符号数的符号位为数值

└←长度应与有符号数相同(便于运算)

小数点—用不同格式表示，类型有定点格式、浮点格式

(点&位置隐含表示) (有/无符号整数) (实数/纯小数)

编码—可为原码/补码/移码/无符号编码 ←便于运算实现

思考①：非数值数据怎么表示？

思考②：机器数11011011的值是多少？

表示要素：进制&格式&编码，格式含小数点、编码含符号

*数据的表示：

硬件需求—每种数据类型只有一种长度 ←硬件定长运算

表示结果—同一类数据可有多种数据类型 ←提高存储效率

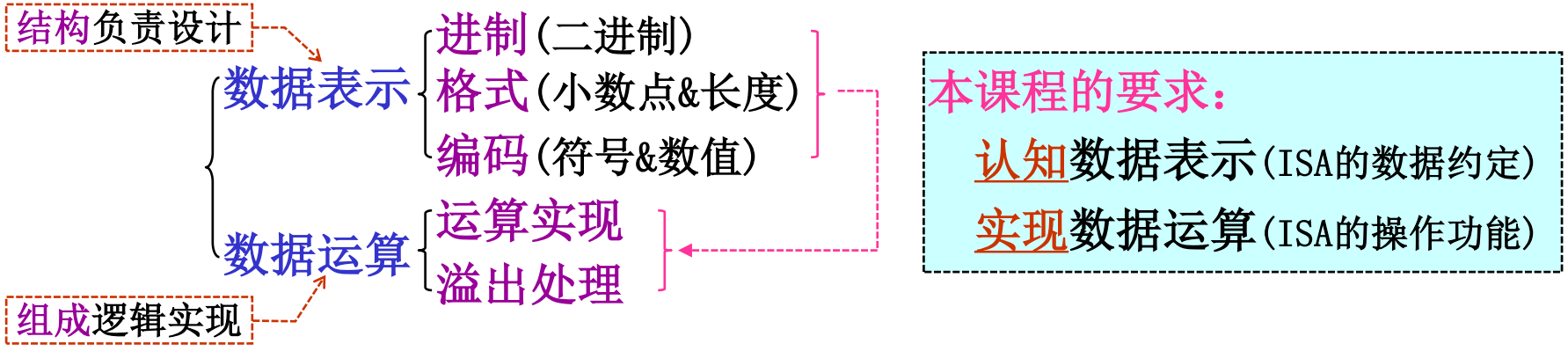
└←表示格式中含长度，如int、short

*数据的运算：

运算实现方法— 基于数据表示(进制/格式[含长度]/编码)进行

溢出处理方法— 硬件检测/通知(高效支持)， 软件处理(灵活性好)

*表示与运算的关系：

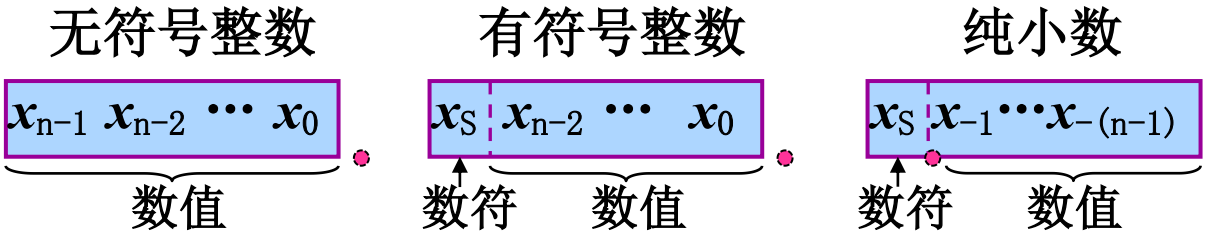


二、整数的表示

1、定点表示方法

*表示方法：数据中的小数点位置固定不变

*表示格式：



*表数范围：（设数码长度为n位）

编码 \ 类型	自然数(无符号)	整数(有符号)	纯小数(有符号)
原码		$-(2^{n-1}-1)\sim+(2^{n-1}-1)$	$-(1-2^{-(n-1)})\sim+(1-2^{-(n-1)})$
补码		$-2^{n-1}\sim+(2^{n-1}-1)$	$-1\sim+(1-2^{-(n-1)})$
无符号编码	$0\sim2^n-1$		

2、整数的表示

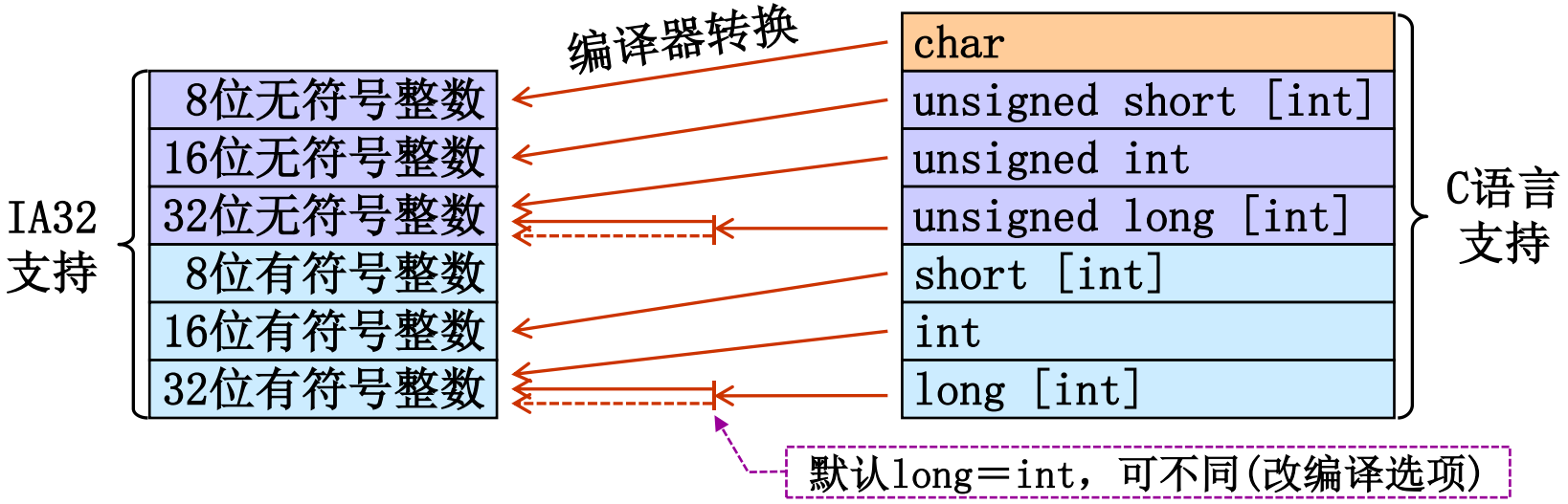
指二进制整数的表示　　←BCD数∠整数(运算方法不同)

*表示方法：无符号整数— 二进制、定点格式、无符号编码

有符号整数— 二进制、定点格式、补码 ←为什么？

*硬件支持的整数类型：取决于ISA的约定，常有多种(长度不同)

例1—IA32(Intel Architecture 32bit)支持6种整数类型



例2—IA32中，若int y=-1；unsigned short n=5；求y、n的机器数

y=FFFFFFFFH，n=0005H

3、整数的类型转换

应用需求—操作数长度可不相同

如：C语言中，short si; int y,z=y+si;

又如：指令R2←R2+imm中，REG为32位，格式中imm为8位

硬件限制—源操作数长度须相同，否则先转换类型

思考①：谁负责转换？ 编译器负责软件需求转换，CPU负责硬件需求转换

***类型转换方法：**

长度相同时—无需运算，解释为新类型

如unsigned short usi=0xFFFE; short si=usi;　FFFEH　si=?

长度减少时—截断运算，仅保留低位　←适合有模运算

如unsigned int uy=0x12345678; short si=uy;　si=?

长度增加时—位扩展运算，保持真值不变　←保持数学含义

思考②：编译器如何实现转换？

如unsigned short usi=2; short si=+2,sj=-2;

unsigned int ua=usi,uy=si; int z=sj;

ua=? uy=? z=?　ua=2，uy=2，z=0xFFFFFFFE

*位扩展运算：（硬件转换，功能为机器数的位数增加、真值不变）

运算方法一 零扩展为机器数高位补0 ←无符号数扩展

符号扩展为机器数高位补符号位 ←有符号数扩展

	D=01101	D=11101
零扩展	Q=000 01101（ 13）	Q=000 11101（ 13）
符号扩展	Q=000 01101（+13）	Q=111 11101（-13）

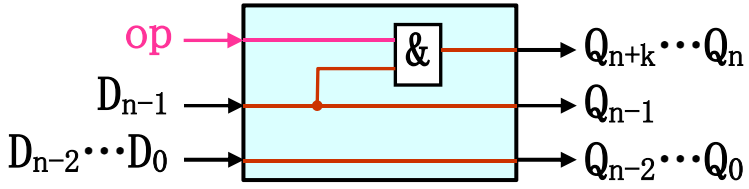
用补码表示

运算实现一设入端为$D_{n-1}\cdots D_0$、出端为$Q_{n+k}\cdots Q_0$，控制端为op

（0/1-零扩展/符号扩展）

输出函数：$Q_{n-1}\sim Q_0=D_{n-1}\sim D_0$，$Q_{n+k}\sim Q_n=op\cdot D_{n-1}$

内部电路：



思考：若只有符号扩展功能，电路(引脚&内部)如何组织？

三、实数的表示

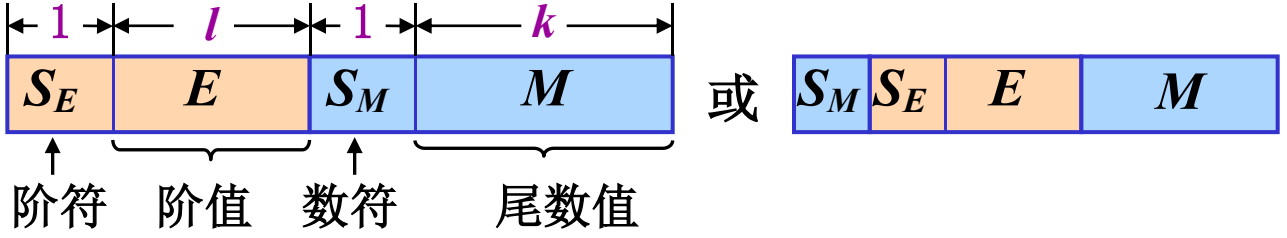
1、浮点表示方法

*表示方法：数据中的小数点位置用阶表示

即实数写成$M\times R_M^{E}$，尾数$M=\pm 0.m_{-1}\cdots m_{-k}$、基为$R_M$；

阶$E=\pm e_{l-1}\cdots e_0$、基为R_E

*表示格式：R_E及R_M隐式表示(R_E=2、通常R_M=2)，其余显式表示



例1—浮点数表示格式中，阶在机器数高位，若尾数为9位、阶码为5位，尾数及阶均用补码表示，写出-54的浮点机器数

解：$-54=(-110110)_2=-0.11011000\times 2^{+0110}$，

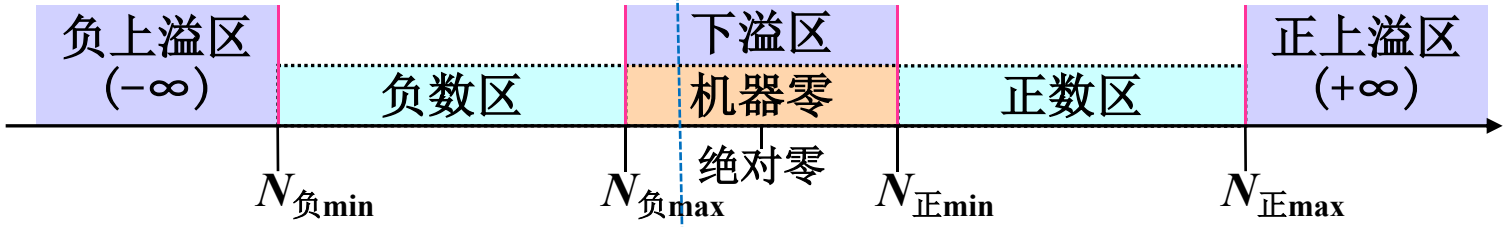
浮点机器数为 00110 100101000

例2—若浮点数用8位尾数、5位阶表示，填充下表空白处

阶	尾数	实数	浮点数的真值	阶码	尾数码
原码	原码	-0.0010111	-0.1011100×2^{-10}	1 0010	1 1011100
			$+0.1011010\times2^{-110}$	1 0110	0 1011010
移码	补码	-10101.11	$-0.1010111\times2^{+101}$	1 0101	1 0101001
			$-0.0011000\times2^{-1011}$	0 0101	1 1101000

练习—若浮点数的尾数用7位补码表示，阶用5位移码表示，写出-54的浮点机器数，及浮点机器数645H的真值

*表数范围与精度：设R_M=2、尾数及阶的值为m位和e位



表数范围— $N_{正max}=-N_{负min}=(1-2^{-m})\times2^{+1\cdots1}$，

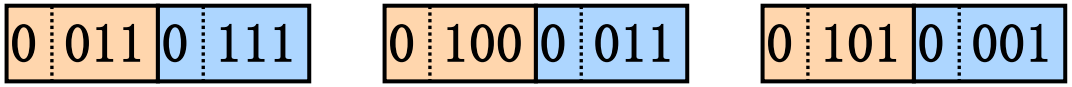
$N_{正min}=-N_{负max}=2^{-m}\times2^{-1\cdots1}$ 受限于 e的位数

表数精度— $\leqslant m$位(首位=0时小于) 受限于m的位数

2、浮点数的规格化　　—使表数精度最大化

表数精度例：$+111.0=+0.111\times2^{3}=+0.0111\times2^{4}=0.00111\times2^{5}$

机器数：



真　值：　+111.0　　+110.0　　+100.0

*规 格 化 数：　尾数的最高数值位为1，即 $0.5\leqslant|M|<1$

*规格化操作：　（浮点数→规格化数）

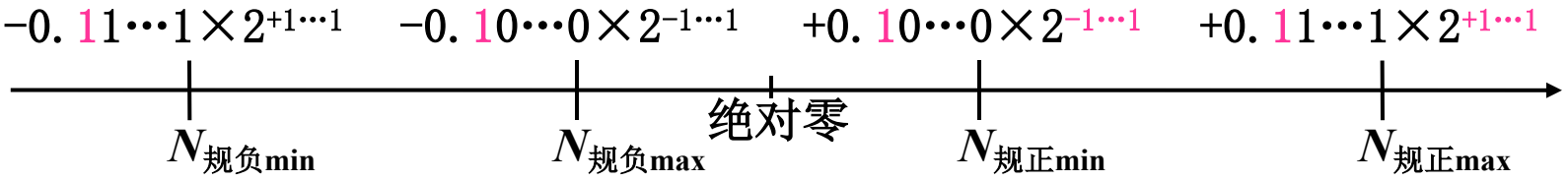
左规—尾数左移一位，阶码减一

右规—尾数右移一位，阶码加一

例3—填充下表空白处

非规格化浮点数	$+1.0111\times2^{+010}$	$-0.00010\times2^{+010}$	$+1011.1\times2^{+010}$
规格化操作	右规1次	左规3次	右规4次
规格化浮点数	$+0.10111\times2^{+011}$	-0.10000×2^{-001}	$+0.10111\times2^{+110}$

*规格化数的数码特征：　(规格化操作的结束条件)



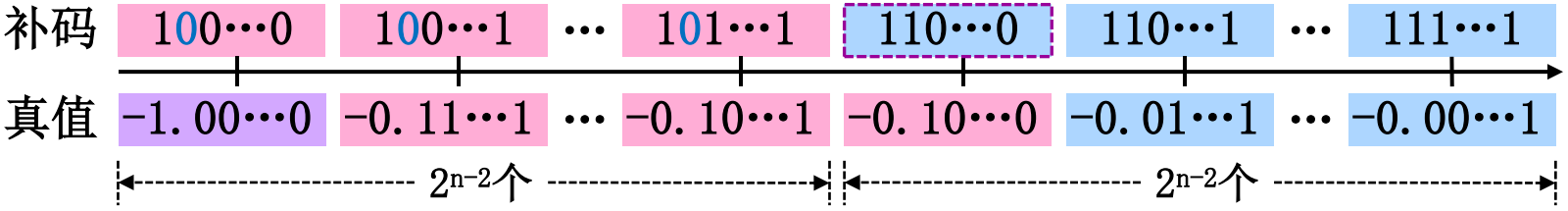
编码格式		规格化数							
		最小负数		最大负数		最小正数		最大正数	
阶	尾数	阶	尾数	阶	尾数	阶	尾数	阶	尾数
原码	原码	0111	111111	1111	110000	1111	010000	0111	011111
移码	补码	1111	100000	0000	101111	0000	010000	1111	011111

补码、移码多表示1个负数

原码尾数— 最高数值位为1

补码尾数— 最高数值位与符号相反　←便于实现、不影响表示精度

(-0.5可再左规)



3、实数的表示

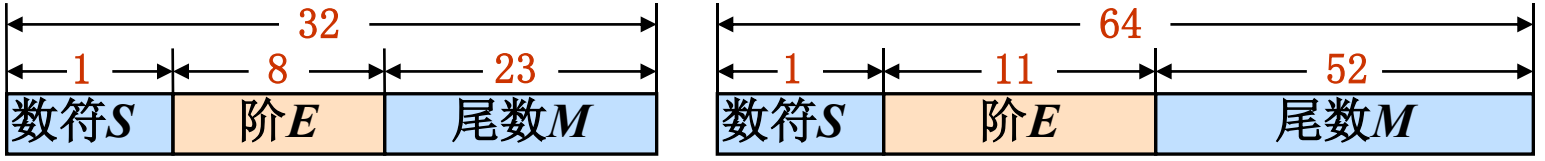
*表示方法：二进制、浮点格式、尾数及阶单独编码

*硬件支持的浮点数类型：

取决于ISA的约定，均为IEEE 754标准(有单精度/双精度2种类型)

4、IEEE 754标准

*表示格式：有单精度、双精度两种　　←数符与尾数分离



单精度浮点表示格式　　双精度浮点表示格式

*编码方式：

数制—M和E均为二进制(即$R_M=R_E=2$)

码制—M为原码定点小数，改变了点位置(表示规格化/非规格化数)

E为移码定点整数，改变了偏移值(区分规格化/非规格化数)

*阶的码制：采用余127码和余1023码

余X码—指偏移值为X的移码　　←移码＝真值＋偏移值

标准移码：E＝阶＋2^{8-1}＝阶＋128　　←可称余128码

余127码：　E＝阶＋($2^{8-1}-1$)＝阶＋127

表示约定— 1≤E≤254表示规格化数的阶(即-126≤阶≤+127)

E＝0及255表示其它特征数(如非规格化数等)

*尾数的码制：采用原码编码，规格化/非规格化数有所不同

非规格化尾数—M的真值＝±0.$m_{-1}\cdots m_{-23}$，　←m_{-1}可以＝0

M的编码＝$Sm_{-1}\cdots m_{-23}$，表示精度≤23位

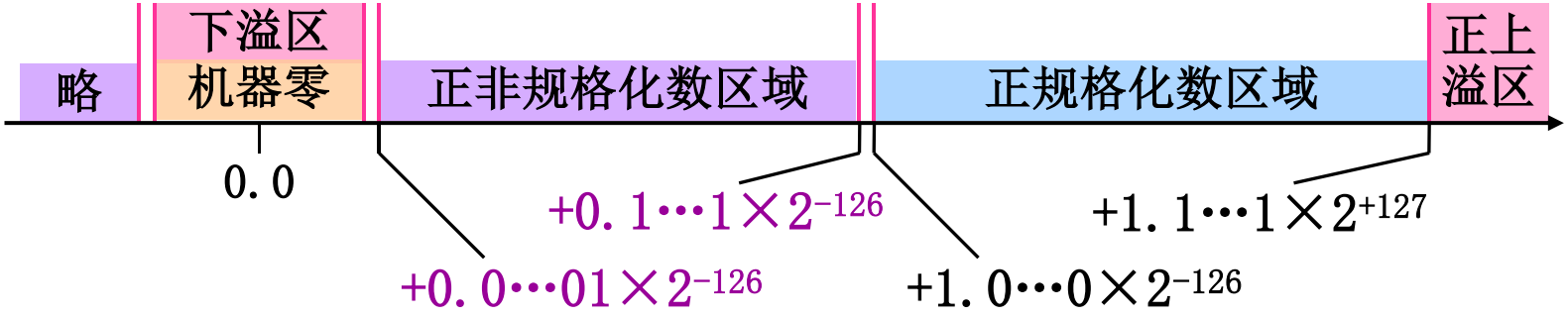
规 格 化尾数—M的真值＝±1.$m_{-2}\cdots m_{-24}$，　←0.1$m_{-2}\cdots m_{-23}$的扩展

M的编码＝$Sm_{-2}\cdots m_{-24}$，表示精度＝24位

*表示特征：（以单精度格式为例）

表示约定—

格式参数	表示的真值N	说明
$E=0$，且$M=0$	$N=\pm 0$	机器零(下溢区)
$E=0$，且$M\neq 0$	$N=(-1)^S\times 2^{-126}\times 0.M$	非规格化数
$1\leqslant E\leqslant 254$	$N=(-1)^S\times 2^{E-127}\times 1.M$	规格化数
$E=255$，且$M=0$	$N=(-1)^S\times\infty$	±无穷大(上溢区)
$E=255$，且$M\neq 0$	$N=$NaN	非数值



表示特征— 可表示溢出区域，可减少下溢区大小，可表示异常(如负数开根等)

例4—求-11/128的IEEE 754单精度浮点数的机器码

解：$-11/128=(-1011)_2\times2^{-7}=(-0.1011)_2\times2^{-3}$

$=[-4]_{移}-1$

$=(-1.011)_2\times2^{-4}=(-1.011)_2\times2^{123-127}$

机器数为：

1	0111 1011	0110 0000 0000 0000 0000 000

例5—求IEEE 754单精度浮点数CC968000H的真值N

解：CC968000H＝1 10011001 00101101000000000000000，规格化数

阶＝10011001B－(10000000-1)B＝+00011010B＝+26

尾数＝1.00101101B＝1.17578125

∴$N=(-1)^1\times1.17578125\times2^{+26}=-1.17578125\times2^{+26}$

练习—写出-180的IEEE 754单精度浮点数的机器码

※数值数据的表示小结：

二进制，定点/浮点格式，某种/两种编码，几种长度

作业2-2：P86起10、14、15(2)、17(2)、18(1)

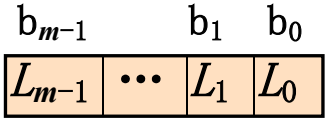
四、非数值数据的表示

1、逻辑数的表示

*数学特征：值域为{真,假}，运算为与(AND)、或(OR)等

*表示方法： 二进制、向量格式、各个数独立编码(逻辑编码)

└←提高存储效率



*硬件支持的类型： 常有多种(长度不同、各自=整数)

*逻辑数的运算：

运算类型—与、或、非等

运算方法— 按位运算　　如C语言中&、|、~等，05H|03H=07H

说明：C语言中&&、‖是用指令串实现的，如a=(i>5 && j<6)编译为if (i<=5) a=0; else if (j>=6) a=0; else a=1;

运算实现—复用整数运算部件(输出端增设选择器)

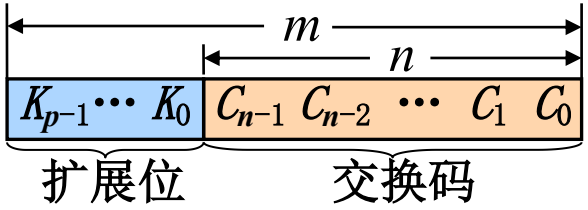
逻辑数常隐含表示(含在整数中)

2、字符的表示

指字符的交换码在存储/运算时的编码表示，即字符的内码

*表示方法： 二进制、定点格式、无符号编码

*硬件支持的类型： 常有几种字符，每种字符长度固定(仅1种)



└←$m=k\times$MEM单元长度

└←可用扩展位或环境变量标识

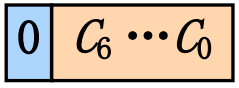
示例1—常见的字符表示 (假设MEM按字节编址)

字符集种类	交换码长度	内码长度	占单元数
ASCII码	7	8位= 7+1 (K_0=0)	1个
GB2312-80码	14	16位=14+2	2个
Unicode码	16或32	2B(UTF16)，1B~6B(UTF-8)	1~6个

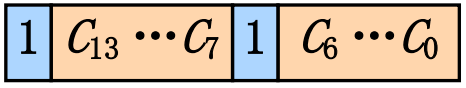
示例2—同时支持ASCII及汉字的字符表示

←无二义性

ASCII字符的表示：



汉字的表示：



*字符的运算：

运算类型—关系运算(A＝B、A≥B等)　←结果为布尔值(真/假)

运算方法— 先比较大小、再形成结果

运算实现— ①通过减法运算(无符号减法)，产生结果标志(Flag)

ZF—Zero Flag
CF—Carry Flag

所需标志：ZF—结果为零时＝1，否则＝0

CF—结果有借位时＝1，否则＝0

②通过逻辑操作(针对结果标志)，形成运算结果

关系运算		A≥B	A≤B	A>B	A<B	A=B
实现	①减法运算	A－B，产生标志(如ZF及CF)				
	②逻辑操作	$\overline{CF}$	CF+ZF	$\overline{CF+ZF}$	CF	ZF

*字符串的表示：

硬件—仅支持字符表示及关系运算　←支持串表示性/价太低

软件—用特定字符表示串结束，串操作转换为字符操作的循环

3、其他类型数据的表示

类型—声音、图像、图形等

*表示策略：硬件支持简单数据类型的表示，←关注性/价

软件负责数据及运算的类型转换

※非数值数据的表示小结：

二进制，定点/向量格式，某种编码，几种长度

- 二进制 └←无符号编码，适于简单数据
- 格式 └←各元素占1位/多位(布尔数/像素)，适于复杂数据
- 编码
- 长度

※非数值数据的运算小结：

逻辑运算—按位的与/或/非等

关系运算—减法运算＋逻辑操作

特殊运算—饱和运算等，如 $x+5=\begin{cases} x+5 & \text{结果}<255 \\ 255 & \text{结果}\geq 255 \end{cases}$

五、数据表示举例

*数据表示的常见结果：

基于使用频率

数据 \ 表示		进制	表示格式	编码方式	长度	运算
数值数据	整数	二	定点(无符号)	无符号编码	几种	算术
		二	定点(有符号)	补码	几种	
		十①	定点(有符号)	BCD码	几种	算术＋校正
	实数	二	浮点(IEEE 754)	尾数原码、阶移码	两种	浮点
非数值数据	逻辑数	二	位向量	逻辑编码	几种	逻辑
	字符	二	定点(无符号)	无符号编码	一种	关系②
	像素	二	定点或向量	无符号编码	几种	饱和等

注：①BCD数≠整数表示　②关系运算用减法运算及逻辑操作实现

ARMv8的数据表示— 8/16/32/64b整数，16/32/64b浮点数，64/128b向量(SIMD数)

包含逻辑数、字符

IA32的数据表示— 整数，浮点数，指针，BCD数，向量，位域

*数据表示与硬件参数的关系：　（ISA研究内容，了解）

数据类型的长度—为2^m位，长度有多种　←便于数据组合

存储单元长度w—即存储器的编址单位

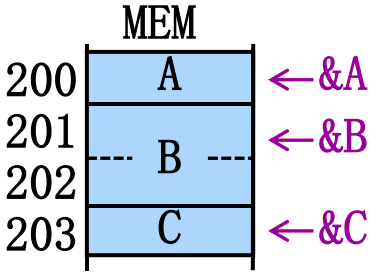
性能指标：存储效率、访问效率

└←侧重点—←┴←可以优化(＞1个单元/次)

组织结果：w＝min{所有数据类型的长度}，通常＝字节

数据存放：连续的x个存储单元中($x \geq 1$)，

地址用最小的单元地址表示



机器字长n—即ALU的宽度

组织结果：　n＝max{所有整数的长度}

※数据表示的学习目标：

认知数据的表示方法，为数据运算的实现打好基础

（真值-机器数的转换）　（基于表示→设计部件）

测试—IA32中，MEM按字节编址，数据在MEM中采用小端方式存放。下列C语句中，数据从2#存储单元起连续存放，写出存放结果

```
char a='A'; //ASCII码为41H
int b=-36;
short c=b, d=a&c;
float e=40;
```

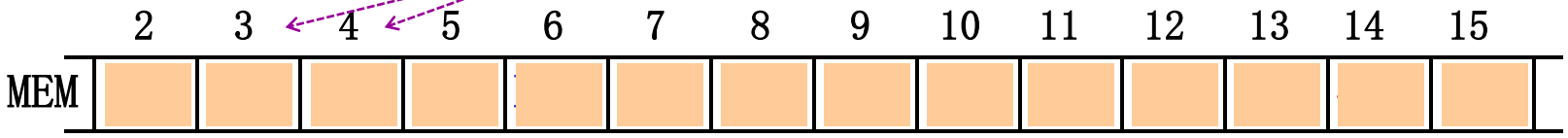
小端方式—数据按字节顺序存放时，地址最小的单元存放最低有效字节

例：数据$B_3B_2B_1B_0$放在40#～43#单元中时，40#存放B_0，41#存放B_1，…

数据表示：①机器数长度分别为 个字节；

②编码为 、 、 、 、

数据存放结果：



§2.3 定点数的运算

回顾逻辑部件

※主要内容：有/无符号的加减、移位、乘法的运算规则、部件组织

一、定点加减运算

运算类型—补码加减、无符号加减，需判断溢出

（原码加减/移码加减不考）

1、补码加减运算

(1)运算方法

数据用补码表示→结果为补码

二进制加法

*加法：$[A+B]_{补}=[A]_{补}+[B]_{补}$ ←A+M+B+M (mod M)=A+B+M (mod M)

*减法：$[A-B]_{补}=[A+(-B)]_{补}=[A]_{补}+[-B]_{补}$

例1—设$[A]_{补}$=11101，$[B]_{补}$=10111，求$[A-B]_{补}$

解：$[-B]_{补}$=01001，

$[A-B]_{补}=[A]_{补}+[-B]_{补}$

=11101+01001=00110

(2)逻辑实现

(3)溢出判断

```
  11101   [-3]补
+ 01001   [+9]补
-------
1 00110   [+6]补
└→进位丢弃(有模运算)
```

例2—设A=-01010，B=+01101，求$[A+B]_{补}$、$[A-B]_{补}$

解：$[A]_{补}$=110110，$[B]_{补}$=001101，$[-B]_{补}$=110011，

$[A+B]_{补}=[A]_{补}+[B]_{补}$=110110+001101= 000011， 符号对

$[A-B]_{补}=[A]_{补}+[-B]_{补}$=110110+110011= 101001，符号对

	110110	$[-10]_{补}$		110110	$[-10]_{补}$
+	001101	$[+13]_{补}$	+	110011	$[+13]_{补}$
1	000011	$[+03]_{补}$	1	101001	$[-23]_{补}$

符号与数值一起运算的正确性(结果不溢出时)证明— (枚举法)

3+2及3-(-2)		3+(-2)及3-2		-3+2及-3-(-2)		-3+(-2)及-3-2	
0011	$([+3]_{补})$	0011	$([+3]_{补})$	1101	$([-3]_{补})$	1101	$([-3]_{补})$
+ 0010	$([+2]_{补})$	+ 1110	$([-2]_{补})$	+ 0010	$([+2]_{补})$	+ 1110	$([-2]_{补})$
0 0101	$([+5]_{补})$	1 0001	$([+1]_{补})$	0 1111	$([-1]_{补})$	1 1011	$([-5]_{补})$

结论：实际结果＝理论结果！

*运算特征：减法可以不比较大小，符号与数值可以一起运算

练习—整数用8位补码表示，A=+15，B=+24，求$[A+B]_{补}$、$[A-B]_{补}$

(2) 逻辑实现

*运算方法特点：加法—$[A]_{补}+[B]_{补}=[A]_{补}+\overline{[B]_{补}}+0$

减法—$[A]_{补}+[-B]_{补}=[A]_{补}+\overline{[B]_{补}}+1$

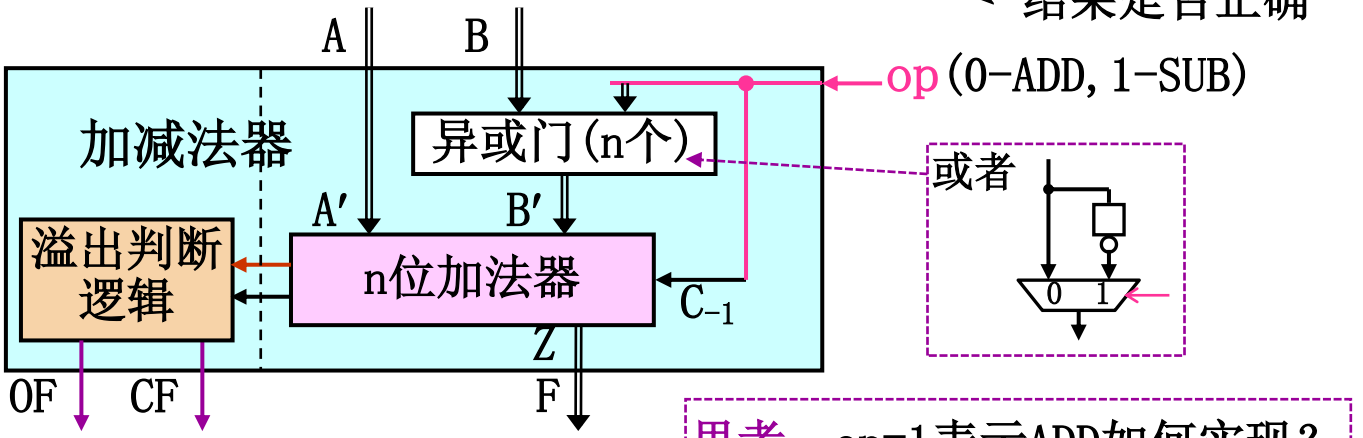
*实现思路：可统一处理，用加法器运算、使$C_{-1}=0/1$(加法/减法时)

即$[A\pm B]_{补}=a_{n-1}a_{n-2}\cdots a_0+b'_{n-1}b'_{n-2}\cdots b'_0+C_{-1}$，

其中 $b'_i=b_i\oplus op_{SUB}$、$C_{-1}=op_{SUB}$ ←$op_{SUB}=1$时表示减法

*加减法器组成：运算处理(加法器及异或门)＋溢出判断(产生OF及CF)

└←结果是否正确



OF—Overflow Flag
CF—Carry Flag

思考：op=1表示ADD如何实现？

(3) 溢出判断　　--判断结果正确性

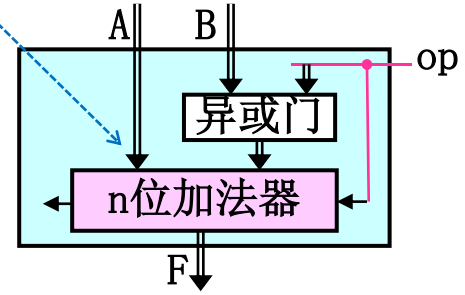
例3—分析4位补码加减运算结果溢出时的符号特征

5+2或5-(-2)	-1+(-4)或-1-(+4)	5+4或5-(-4)	-5+(-4)或-5-(+4)
0101($[+5]_{补}$)	1111($[-1]_{补}$)	0101($[+5]_{补}$)	1011($[-5]_{补}$)
+ 0010($[+2]_{补}$)	+ 1100($[-4]_{补}$)	+ 0100($[+4]_{补}$)	+ 1100($[-4]_{补}$)
0 0111($[+7]_{补}$)	1 1011($[-5]_{补}$)	0 1001($[-7]_{补}$)	1 0111($[+7]_{补}$)

*溢出特征：①同号相加或异号相减时，结果与被加(减)数异号；

②最高位、次高位的进位中，只有1个有进位

*基于加减法器的溢出判断：



设A＝$a_{n-1}a_{n-2}\cdots a_0$，B＝$b_{n-1}b_{n-2}\cdots b_0$，F＝$f_{n-1}f_{n-2}\cdots f_0$，

op＝0/1(加/减)

则溢出标志OD的逻辑为

$$OF=\overline{op}(\overline{a_{n-1}}\,\overline{b_{n-1}}f_{n-1}+a_{n-1}b_{n-1}\overline{f_{n-1}})+op(\overline{a_{n-1}}\,\overline{b_{n-1}}f_{n-1}+a_{n-1}\overline{b_{n-1}}\,\overline{f_{n-1}})$$

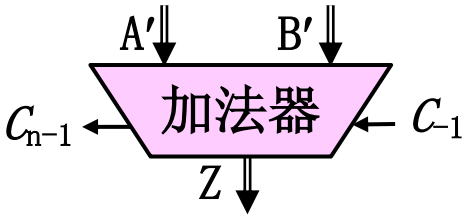
$$=(a_{n-1}\odot(op\oplus b_{n-1}))(a_{n-1}\oplus f_{n-1})$$

特点—逻辑复杂，可优化(基于内部加法器判断时与op无关)

*基于内部加法器的溢出判断：

设$A'=a_{n-1}a_{n-2}\cdots a_0$、$B'=b'_{n-1}b'_{n-2}\cdots b'_0$，

$Z=z_{n-1}z_{n-2}\cdots z_0$



溢出特征— 正溢出 $(a_{n-1}=b'_{n-1}=0)$时，$z_{n-1}=1$、$C_{n-1}=0$、$C_{n-2}=1$

（参考上页例） 负溢出 $(a_{n-1}=b'_{n-1}=1)$时，$z_{n-1}=0$、$C_{n-1}=1$、$C_{n-2}=0$

判断方法①—用1位符号位判断

$$OF=\overline{a_{n-1}}\,\overline{b'_{n-1}}z_{n-1}+a_{n-1}b'_{n-1}\overline{z_{n-1}}=(a_{n-1}\oplus z_{n-1})\,(b'_{n-1}\oplus z_{n-1})$$

判断方法②—用进位位判断

$$OF=C_{n-1}\oplus C_{n-2}$$

例4—设$[A]_{补}$=1010，$[B]_{补}$=0011，$[A+B]_{补}$、$[A-B]_{补}$溢出否?

解：$[A+B]_{补}$=1010+0011=0 1101，$[A-B]_{补}$=1010+1101=1 0111

	$[A+B]_{补}$溢出判断	$[A-B]_{补}$溢出判断
使用方法①	$OF=(1\oplus1)\,(0\oplus1)=0$	$OF=(1\oplus0)\,(1\oplus0)=1$
使用方法②	$OF=0\oplus0=0$	$OF=1\oplus0=1$

判断方法③—用2位符号位判断

变形补码：采用2个符号位的补码$[Z]_{变补}=z_nz_{n-1}z_{n-2}\cdots z_0$，

z_nz_{n-1}表示符号(00为正数、11为负数)

依据：正溢出时$z_nz_{n-1}=01$，负溢出时$z_nz_{n-1}=10$

($C_{n-1}=0$、$C_{n-2}=1$)　　($C_{n-1}=1$、$C_{n-2}=0$)

溢出逻辑：$OF=z_n\oplus z_{n-1}$

例5—设A=-110，B=+011，$[A+B]_{变补}$、$[A-B]_{变补}$是否溢出?

解：$[A]_{变补}$=11 010，$[B]_{变补}$=00 011，$[-B]_{变补}$=11 101

$[A+B]_{变补}$ = 11 010　　$[A-B]_{变补}$ = 11 010

+ 00 011　　+ 11 101

11 101　　10 111

OF=1⊕1=0，不溢出　　OF=1⊕0=1，溢出

练习—设A=+15，B=+24，补码含5个数值位，判断A+B、A-B结果是否溢出(方法不限)

2、无符号加减运算

*运算方法：加法—$[A+B]_{无}=[A]_{无}+[B]_{无}=[A]_{无}+[B]_{无}+0$

减法—$[A-B]_{无}=[A]_{无}+[B]_{补数}=[A]_{无}+\overline{[B]_{无}}+1$

*逻辑实现：

实现思路—与补码加减法相同

加减法器组成—与补码加减法器相同

同一个加减法器可实现有/无符号的加/减运算

思考①：加减法器怎知运算有/无符号？

无需区分符号（运算结果相同）

*溢出判断：--判断结果正确性

溢出特征—加法时有进位($C_{n-1}=1$)，

减法时有借位($C_{n-1}=0$)

10+9或10-7	7+6或7-10
1010	0111
+ 1001	+ 0110
1 0011	0 1101

判断方法—$CF=C_{n-1}\oplus op_{SUB}$ ←$op_{SUB}=1$时表示减法

思考②：加减法器溢出判断区分有/无符号(运算不区分)。

使用者同时仅一种运算，加减法器如何提供支持？

同时产生OF及CF（基于运算选用）

~~方案2~~：增加引脚S(表示有/无符号)，标志仅OF(逻辑用S选择)，比同时产生好吗？

OF仅表示有符号溢出，CF仅表示无符号溢出

3、加减法器的结果标志

*关系运算的实现：

无符号关系运算— ①通过减法运算(无符号减法)， 产生结果标志

②通过逻辑操作(针对CF/ZF)， 形成运算结果

关系运算		A≥B	A≤B	A>B	A<B	A=B
实现	①减法运算	A−B，产生标志(ZF及CF)				
	②逻辑操作	$\overline{CF}$	CF+ZF	$\overline{CF+ZF}$	CF	ZF

有符号关系运算— ①通过减法运算(有符号减法)， 产生结果标志

②通过逻辑操作(针对OF/SF/ZF)， 形成运算结果

OF—Overflow Flag
SF—Signal Flag

SF=1(结果<0时)
或0(结果≥0时)

关系运算		A≥B	A≤B	A>B	A<B	A=B
实现	①减法运算	A−B，产生标志(ZF、OF及SF)				
	②逻辑操作	$\overline{OF \oplus SF}$	$OF \oplus SF+ZF$	$\overline{OF \oplus SF} \cdot \overline{ZF}$	$OF \oplus SF$	ZF

分析—4位补码减法，

(-1)＜4 =1011(-5) OF=0, SF=1

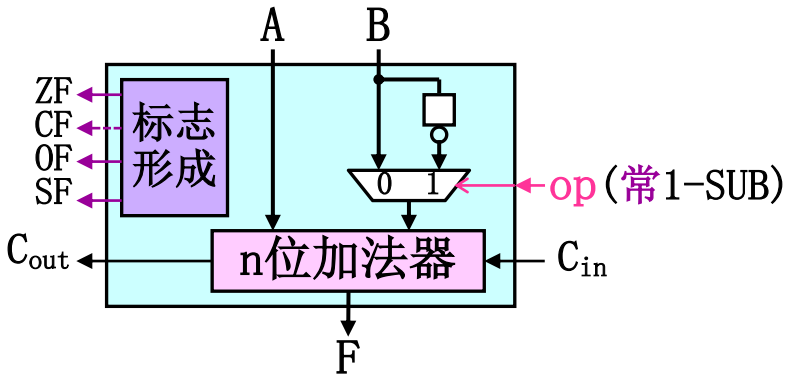
(-5)＜4 =0111(+7) OF=1, SF=0

4≮3 =0001(+1) OF=0, SF=0

5≮(-4) =1001(-7) OF=1, SF=1

*功能集成的加减法器：

①常设置C_{in}引脚 ←便于级联(需带进位加法/借位减法运算)



需求 1122H 11H 22H
+3344H = +33H 及 +44H
+ CF

思考①：左图实现A+B、A-B时的C_{in}取值？

加法：$F=A+B+C_{in}$

减法：C_{in}=1时$F=A+\overline{B}+1-0=A-B-0$，

C_{in}=0时$F=A+\overline{B}+1-1=A-B-1$，

即$F=A-B-\overline{C_{in}}$

思考②：上图的加/减运算功能？

②需产生结果标志位 ←检测溢出(结果正确性)、支持关系运算

ZF—结果为零时＝1、否则＝0， 即$ZF=\overline{F_{n-1}+\cdots+F_0}$

CF—结果有进位/借位时＝1、否则＝0， 即$CF=C_{out}\oplus op_{SUB}$

OF—结果溢出时＝1、否则＝0， 即$OF=C_{out}\oplus C_{n-2}$ 或 …

SF—结果为负时＝1、否则＝0， 即$SF=F_{n-1}$

思考③：CF和OF都表示结果溢出，为什么不合并？ 否则不能同时判断有/无符号(op增加1位)

练习：机器数A＝74H、B＝A3H，A+B、A-B后的标志各是？

A及B分别为有、无符号数时，A＜B、A＝B的运算结果是？

4、原码加减运算　　（不考，了解）

运算特征—符号与数值分开运算，减法需先比较大小

*运算方法：设$[A]_{原}=a_{n-1}\cdots a_0$，$[B]_{原}=b_{n-1}\cdots b_0$，op=0/1(加/减)

(1)判求和/差，$op'=(a_{n-1}\oplus b_{n-1})\oplus op$

(2)求和时，$z_{n-1}=a_{n-1}$，$z_{n-2}\cdots z_0=a_{n-2}\cdots a_0+b_{n-2}\cdots b_0$，$OF=C_{n-2}$

(3)求差时，$z'_{n-2}\cdots z'_0=a_{n-2}\cdots a_0+\overline{b_{n-2}}\cdots\overline{b_0}+1$，　　$OF=0$

够减时($C_{n-2}=1$)，$z_{n-1}=a_{n-1}$，$z_{n-2}\cdots z_0=z'_{n-2}\cdots z'_0$

不够减时($C_{n-2}=0$)，$z_{n-1}=\overline{a_{n-1}}$，$z_{n-2}\cdots z_0=\overline{z'_{n-2}}\cdots\overline{z'_0}+1$

*逻辑实现：加减法器＋辅助电路(判断/暂存结果)

*溢出判断：$OF=\overline{(a_{n-1}\oplus b_{n-1})\oplus op}\cdot C_{n-2}$

5、移码加减运算　　（不考，了解）

运算特征—类似于补码运算(教材P78)

※习题课-1：[2011年研究生入学统考题]

假定在一个8位字长的计算机中运行如下C程序段：

```
unsigned int x = 134, y = 246, z1 = x-y, z2 = x+y;
int m = x, n = y, k1 = m-n, k2 = m+n;
```

若编译器编译时，将8个8位寄存器R1~R8分别分配给变量x、y、m、n、z1、z2、k1、k2。请回答下列问题。

（1）执行上述程序段后，寄存器R1、R5和R6的内容分别是什么？

（2）执行上述程序段后，变量m和k1的值分别是什么？（十进制）

（3）上述程序段涉及带符号整数加/减、无符号整数加/减运算，这四种运算能否利用同一个加法器及辅助电路实现？简述理由。

（4）如何判断带符号整数加/减运算的结果是否发生溢出？上述程序段中，哪些带符号整数运算语句的执行结果会发生溢出？

1）x=10000110B，y=11110110B，z1=x-y=(0)10010000，z2=x+y=(1)01111100，(R1)=86H，(R5)=90H，(R6)=7CH

2）m=86H=-122，n=F6H，k1=m-n=x-y=86H-F6H=90H=-112

4）判断：同号相加或异号相减，结果与被加(减)数异号。仅k2可能溢出(有符号加)，k2=(1)01111100，溢出

二、移位运算

运算类型—逻辑左移、逻辑右移、算术左移、算术右移

（针对无符号数）　　（针对有符号数）

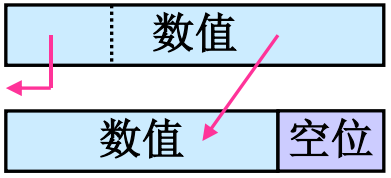
功能：A×2^k或A÷2^k，A为数据、k为移位位数

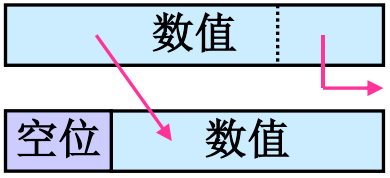
表示：$<<_L$、$>>_L$、$<<_A$、$>>_A$　　←Logical shift及Arithmetic shift

1、逻辑移位(Logical Shift)运算

*运算方法：　机器数整体移位；

移出的数位丢弃，出现的空位补0(无符号数×2^k或÷2^k)





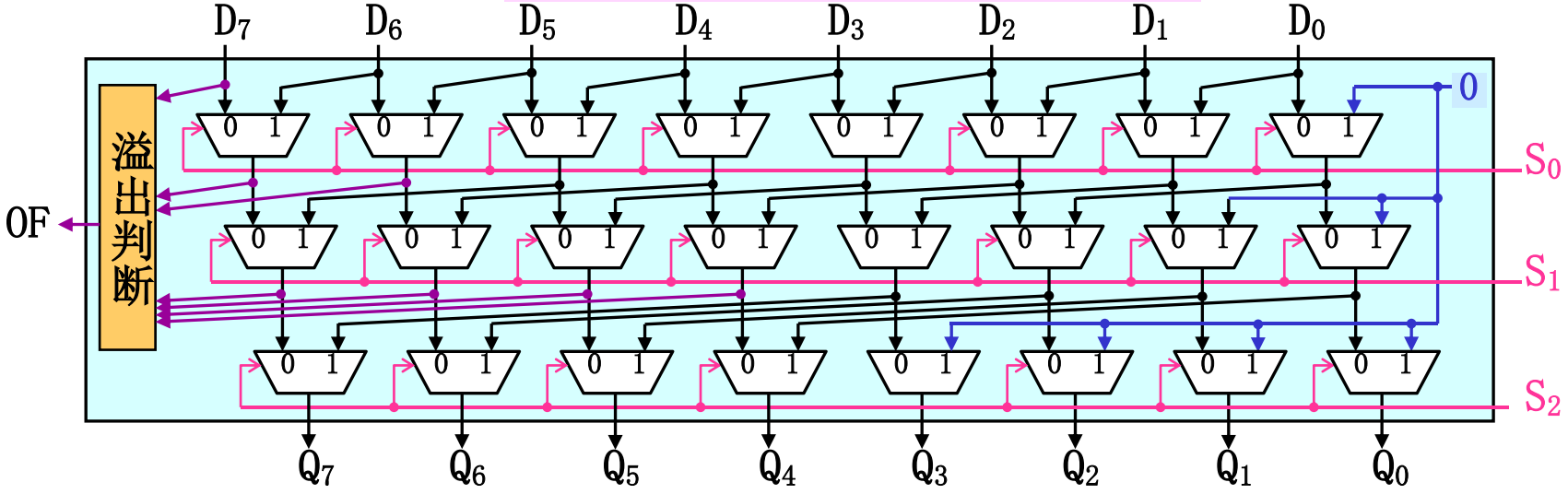
例1—无符号整数的机器码为8位，求57 $<<_L$ 2、57 $>>_L$ 2

解：57=00111001B，57 $<<_L$ 2＝228＝11100100B；

57=00111001B，57 $>>_L$ 2＝ 14＝00001110B

*逻辑实现： 桶形移位器(组合逻辑电路)

功能表：$Q_7 \sim Q_0 = D_7 \sim D_0 \ll_L S_2S_1S_0$



思考：为什么不用移位寄存器实现？ 性能差(移动1位/Clk)

*溢出判断： 设入端为$D_{n-1}\cdots D_0$、控制端为$S_{k-1}\cdots S_0$ ←$k=\log_2 n$

溢出条件—左移时移丢码1 (右移时移丢码1仅影响精度)

判断方法—$OF = S_{k-1}(D''_{n-1}+\cdots+D''_{n/2}) + S_{k-2}(D'_{n-1}+\cdots+D'_{3n/4}) + \cdots + S_0D_{n-1}$

逻辑或

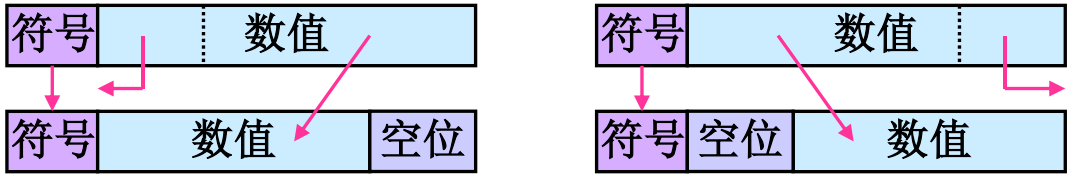
如：n=4时，$OF = S_1(D'_3+D'_2) + S_0D_3$

n=8时，$OF = S_2(D''_7+\cdots+D''_4) + S_1(D'_7+D'_6) + S_0D_7$

2、算术移位(Arithmetic shift)运算

*运算方法：机器数符号位不变、数值位整体移位；

移出的数位丢弃，出现的空位根据编码特征添补



空位添补规则—　(有符号机器数×2^k或÷2^k)

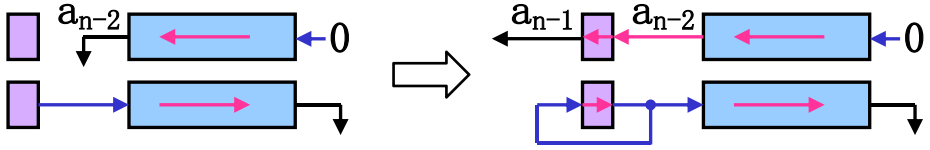
码制	符号位	左移空位添补	右移空位添补
原码	保持不变	补0	补0
补码		补0	补符号位

例2	真值	原码	补码	真值	原码	补码
	+2	00010	00010	−2	10010	11110
算术左移1位(Y<<$_A$1)	+4	00100	00100	−4	10100	11100
算术右移1位(Y>>$_A$1)	+1	00001	00001	−1	10001	11111

练习—整数用8位补码表示，设A=+15，B=−25，写出B>>1、A+B<<2的机器数

*逻辑实现：符号与数值位一起移位　←便于复用逻辑移位电路

（不考虑溢出）



*溢出判断：

溢出条件— 左移时移丢(指a_{n-2})与符号位相反的码

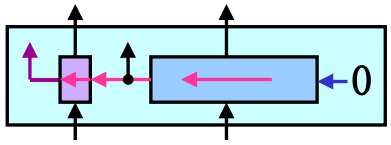
例：连续<<$_A$，0011→0110→1100，1101→1010→0100

+3　+6　−4　−3　−6　+4

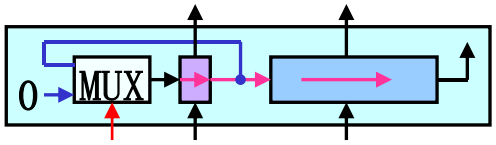
判断方法— 类似于逻辑移位，判断最高位与其余位(n-2位起)

$$OF=D_{n-1}\oplus(S_{k-1}(D''_{n-2}+\cdots+D''_{n/2})+S_{k-2}(D'_{n-2}+\cdots+D'_{3n/4})+\cdots+S_1D_{n-2}))$$

*功能集成的桶形移位器：　支持逻辑移位及算术移位



逻辑左移/算术左移



逻辑右移/算术右移

作业2-3：P87—23、26、27

三、定点乘法运算

运算类型—无符号乘法、补码乘法、原码乘法　（除法不考）
（用于整数）　（用于小数）　←基于数据表示

*机器乘法的实现需求：　—硬件为定长运算

多次累加形成乘积，部分积移位后再相加

*机器乘法的实现方法：　—尽量复用器件

①用循环的加法和右移实现

②加法时，加被乘数或0

③右移时，加法进位、部分积高位、
部分积低位一起右移

加法器位置不变时

```
        1 1 1
×       1 0 1
-------------
      0 0 0      …P0(初值)
    + 1 1 1
    ---------
    0 1 1 1      …P1
  + 0 0 0
  -------
  0 0 1 1 1      …P2
+ 1 1 1
---------
1 0 0 0 1 1    …P3
```

*机器乘法的类型：　—基于每次所乘的位数

1位、2位乘法等(加法器＋移位器)，快速乘法(阵列乘法器)

| 　　└←每次判断2位乘数、加0/A/2A/3A、右移2位，循环n/2次

└←每次判断1位乘数、加0/A、右移1位，循环n次

1、无符号乘法运算　　（只讨论整数一位乘法）

$[A\times B]_{无}=[A]_{无}\times[B]_{无}$，A、B长度须相同，结果应为2n位

有时为n位，如int=int*int

***运算方法：**

$$A\times B=A\times b_{n-1}\times 2^{n-1}+\cdots+A\times b_1\times 2^1+A\times b_0\times 2^0$$

$$=2^n\times\{A\times b_{n-1}\times 2^{-1}+\cdots+A\times b_1\times 2^{-(n-1)}+A\times b_0\times 2^{-n}\}$$

$$=2^n\times\{(A\times b_{n-1}+(\cdots+(A\times b_1+(0+A\times b_0)\times 2^{-1})\times 2^{-1}\cdots)\times 2^{-1})\times 2^{-1}\}$$

n个

部分积初值(为递推所设置)

递推公式— 设初始部分积$P_0=0$

$$P_1=(P_0+A\times b_0)\times 2^{-1}$$

…

$$P_i=(P_{i-1}+A\times b_{i-1})\times 2^{-1}$$　←需带进位右移(加法有进位)

…

$$P_n=(P_{n-1}+A\times b_{n-1})\times 2^{-1}$$　←循环为n次

实质— 循环的判断-加法-移位操作(判$b_{当前}$，+0/A，带进位右移)

例1—设A=111、B=110，用一位乘法求$[A\times B]_{无}$

解：

部分积高位	低位	乘数	说明
000		110	初始部分积P_0=0
+ 000 0 000 000	0	110	当前乘数位=0，$P_{0高3位}$+0 部分积P_1带进位右移1位
+ 111 0 111 011	10	110	当前乘数位=1，$P_{1高3位}$+A 部分积P_2带进位右移1位
+ 111 1 010 101	010		当前乘数位=1，$P_{2高3位}$+A 部分积P_3带进位右移1位

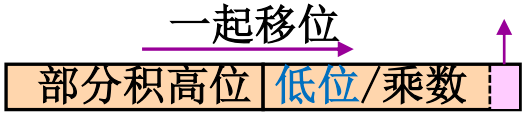
练习—无符号整数用4位无符号编码表示，用无符号一位乘法求$[9\times 6]_{无}$

*运算实现：

思路— ①循环的判断-加法-移位操作

判断	加法（n位）	移位
$b_{i-1}=0$	$P'_{i高n位}=P_{i-1高n位}+0$，$P'_{i低位}=P_{i-1低位}$	$P_i=P'_i$带进位 右移1位
$b_{i-1}=1$	$P'_{i高n位}=P_{i-1高n位}+A$，$P'_{i低位}=P_{i-1低位}$	

②部分积低位放在乘数的空位上 ←节省REG



├→可仅判断REG的最低位 ←实现简单

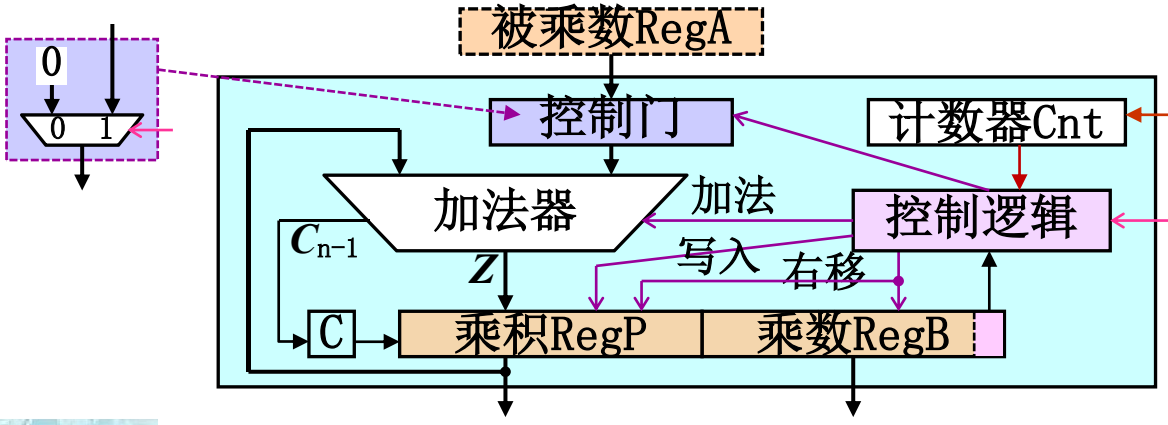
└→需部分积与乘数一起右移

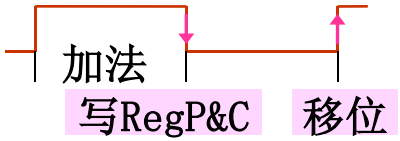
组成— $RegB_0$控制控制门(输出0/RegA)，RegP/RegB一起移位，

└←写入后移位

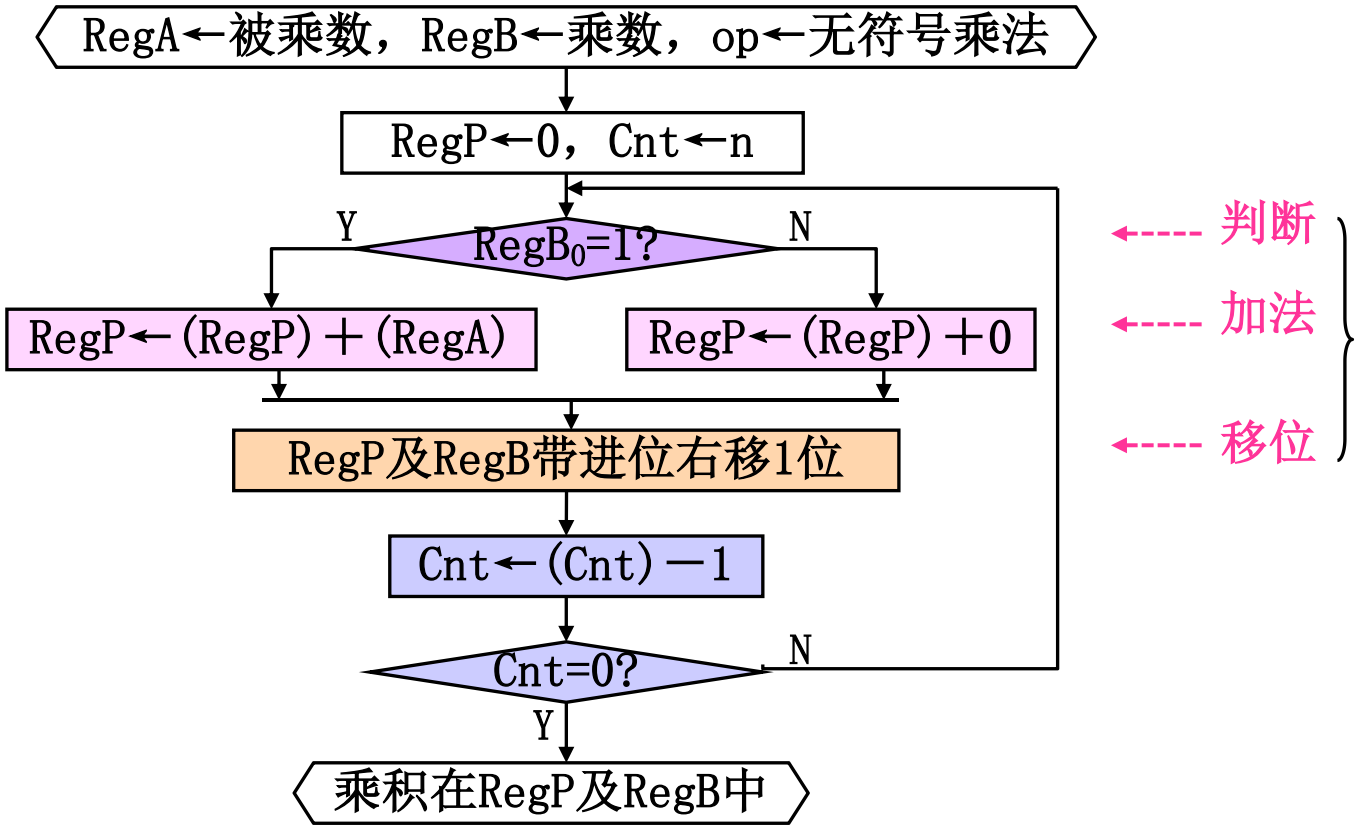
Cnt控制循环(初值=n、为0时结束)

控制逻辑：
控制门—=$RegB_0$
加法器—=1
写　入—=1，CLK↓触发
右　移—=1，CLK↑触发





控制流程—循环的判断-加法-移位操作



***溢出判断：** （基于指令需求[非乘法运算本身]）

乘法结果为2n位时— OF＝0（不会溢出）

乘法结果为 n位时— OF＝z_{2n-1}+…+z_n

硬件按n位产生OF，指令决定是否保存(视为异常)

←高位有1就溢出

2、补码乘法运算　　（只讨论整数一位乘法）

$[A\times B]_{补}=[A]_{补}\times[B]_{补}$，结果应为2n位

***运算方法：**又称Booth算法

设$[B]_{补}=b_{n-1}b_{n-2}\cdots b_0$，则$B=b_{n-1}b_{n-2}\cdots b_0-2^n=-b_{n-1}\times 2^{n-1}+b_{n-2}\cdots b_0$

$b_{n-1}=0$时如此（代入）→⊥←$b_{n-1}=1$时如此（代入）

有$B=-b_{n-1}\times 2^{n-1}+b_{n-2}\times 2^{n-2}+\cdots+b_0\times 2^0$

$=-b_{n-1}2^{n-1}+(b_{n-2}2^{n-1}-b_{n-2}2^{n-2})+\cdots+(b_0 2^1-b_0 2^0)$　←$2^{i-1}=2^i-2^{i-1}$

$=(b_{n-2}-b_{n-1})2^{n-1}+(b_{n-3}-b_{n-2})2^{n-2}+\cdots+(0-b_0)2^0$

$[A\times B]_{补}=[A\times\{(b_{n-2}-b_{n-1})2^{n-1}+(b_{n-3}-b_{n-2})2^{n-2}+\cdots+(0-b_0)2^0\}]_{补}$

$=2^n\times[A\times\{(b_{n-2}-b_{n-1})2^{-1}+(b_{n-3}-b_{n-2})2^{-2}+\cdots+(0-b_0)2^{-n}\}]_{补}$

$=2^n\times[\{(b_{n-2}-b_{n-1})\times A+\{\cdots+\{(b_0-b_1)\times A$

$+\{0+(0-b_0)\times A\}\times 2^{-1}\}\times 2^{-1}\}\cdots\}\times 2^{-1}]_{补}$

（$\times 2^{-1}$共n次）

由于$[A\times 2^{-1}]_{补}=[A]_{补}\times 2^{-1}$，则

$=2^n\times\{[(b_{n-2}-b_{n-1})\times A]_{补}+\{\cdots+\{[(b_0-b_1)\times A]_{补}$

$+\{0+[(0-b_0)\times A]_{补}\}\times 2^{-1}\}\times 2^{-1}\}\cdots\}\times 2^{-1}$

（$\times 2^{-1}$共n次）

递推公式—设$[B]_{补}=b_{n-1}b_{n-2}\cdots b_0$，附加位$b_{-1}=0$，$[P_0]_{补}=0$

$$[P_1]_{补}=\{[P_0]_{补}+[(b_{-1}-b_0)\times A]_{补}\}\times 2^{-1}$$

…

$$[P_i]_{补}=\{[P_{i-1}]_{补}+[(b_{i-2}-b_{i-1})\times A]_{补}\}\times 2^{-1}$$ ←算术右移

…

$$[P_n]_{补}=\{[P_{n-1}]_{补}+[(b_{n-2}-b_{n-1})\times A]_{补}\}\times 2^{-1}$$

实质—循环的判断-加法-移位操作(判$b_{当前}b_{上次}$，+0/+A/-A，算术右移)

例2—$[A]_{补}$=011，$[B]_{补}$=101，用Booth算法求$[A\times B]_{补}$

部分积高位	低位	乘数	b_{-1}	说明
000		101	0	初始：P_0=0，b_{-1}=0
+ 101 101 110	1	10	1	b_0b_{-1}=10，+$[-A]_{补}$ 部分积算术右移1位
+ 011 001 000	11	1	0	b_1b_0=01，+$[A]_{补}$ 部分积算术右移1位
+ 101 101 110	111		1	b_2b_1=10，+$[-A]_{补}$ 部分积算术右移1位

思考：右移时为何不考虑加法的进位？

设B=1 0 1 1 0,

∵循环时2个10间必有01

∴+A、-A交替进行

*运算实现：

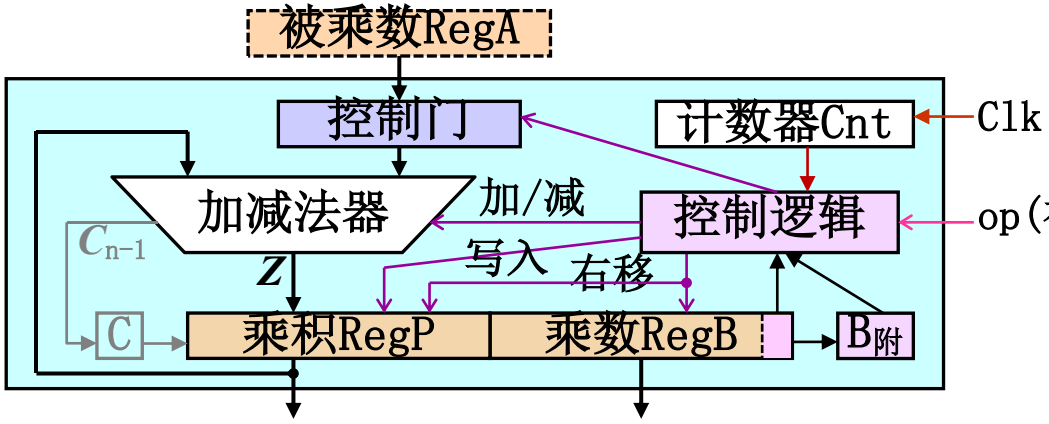
思路— ①循环的判断-加法-移位操作

判断	加法（n位）	移位
$b_{i-1}b_{i-2}$＝00或11	$P'_{i高n位}=P_{i-1高n位}+0$， $P'_{i低位}=P_{i-1低位}$	$P_i=P'_i$算术右移1位
$b_{i-1}b_{i-2}$＝01	$P'_{i高n位}=P_{i-1高n位}+[A]_{补}$， $P'_{i低位}=P_{i-1低位}$	
$b_{i-1}b_{i-2}$＝10	$P'_{i高n位}=P_{i-1高n位}+[-A]_{补}$， $P'_{i低位}=P_{i-1低位}$	

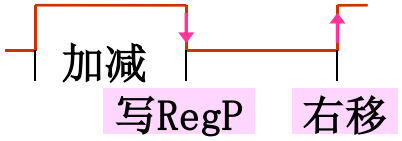
②部分积低位放在乘数的空位上 ←同无符号乘法

组成— 扩展无符号乘法器(增设$B_{附}$、改为加减法器、缺省C)，

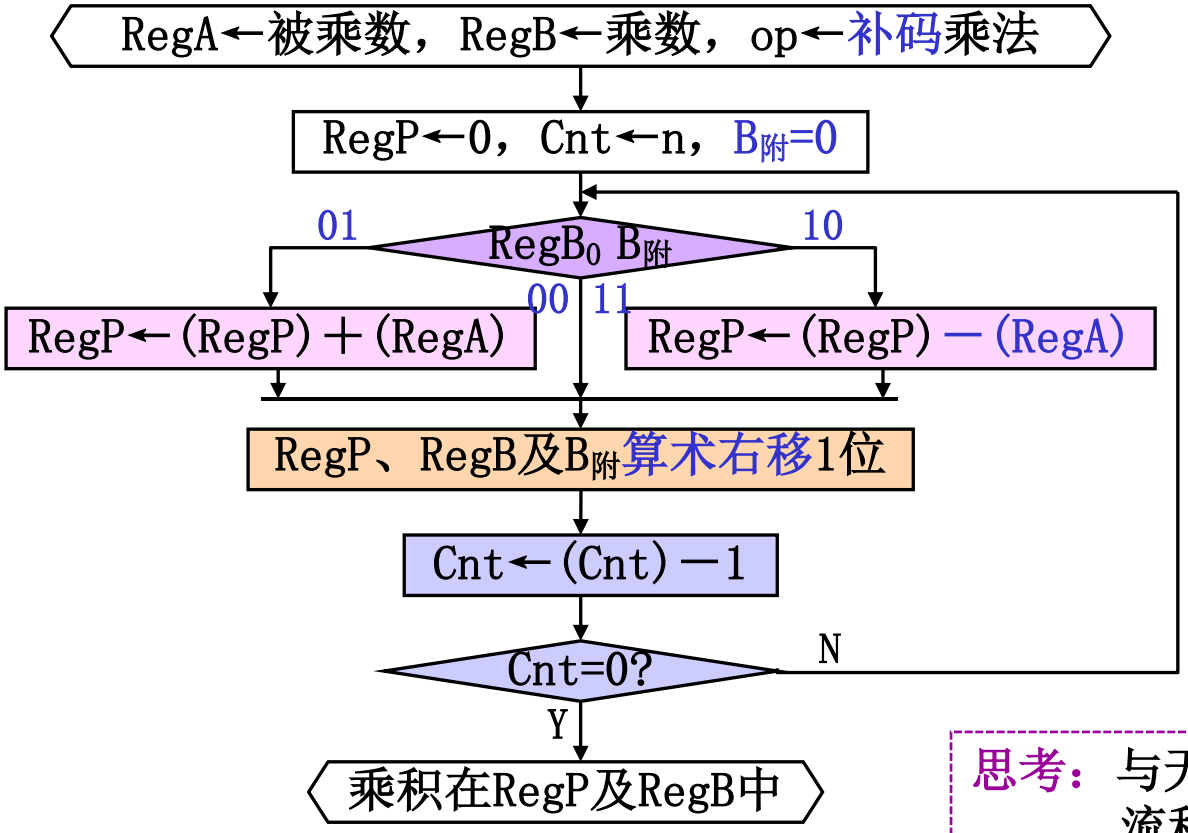
$RegB_0$及$B_{附}$控制控制门及±，RegP/RegB/$B_{附}$一起移位



控制逻辑：
控 制 门—＝$RegB_0 \oplus B_{附}$
加减法器—＝$RegB_0 \cdot \overline{B_{附}}$
写入&右移—＝1



控制流程—循环的判断-加法-移位操作



思考：与无符号乘法控制流程的差别？

*溢出判断：（基于指令需求[非乘法运算本身]）

乘法结果为2n位时— $OF=0$（不会溢出）

乘法结果为 n位时— $OF=\overline{z_{n-1}}(z_{2n-1}+\cdots+z_n)+z_{n-1}(\overline{z_{2n-1}}+\cdots+\overline{z_n})$

3、原码乘法运算　　（只讨论小数一位乘法）

$[A\times B]_{原}=[A]_{原}\times[B]_{原}$，结果为≤2n-1位

*运算方法：设$[B]_{原}=b_{n-1}b_{n-2}\cdots b_0$，$[A\times B]_{原}=z_{2n-2}\cdots z_0$

①求乘积符号，$S_P=a_{n-1}\oplus b_{n-1}$，　　$z_{2n-2}=S_P$

②求乘积数值，$P=|A|\times|B|$，$z_{2n-3}\cdots z_0=P$

乘积数值的递推公式—

$$P_i=(P_{i-1}+|A|\times b_{i-1})\times 2^{-1}$$

←n-1位加法（无符号乘法为n位）

←n-1次循环（无符号乘法为n次）

实质—符号/数值分开运算，数值相乘为无符号乘法（n-1位）

例3—设$[A]_{原}$=0.111、$[A]_{原}$=1.110，求$[A\times B]_{原}$

解：|A|=0.111，|B|=0.110；

$S_P=0\oplus 1=1$；$P=|A|\times|B|=0.101010$（同例1[循环3次]），

$[A\times B]_{原}=1.101010$

*运算实现：

思路—①计算并暂存结果符号

②循环n-1次的判断-加法-移位操作

判断	加法(n-1位)	移位
$b_{i-1}=0$	$P'_{i高n-1位}=P_{i-1高n-1位}+0$，　$P'_{i低位}=P_{i-1低位}$	$P_i=P'_i$带进位右移1位
$b_{i-1}=1$	$P'_{i高n-1位}=P_{i-1高n-1位}+\|A\|$，$P'_{i低位}=P_{i-1低位}$	

③部分积低位放在乘数的空位上　←同无符号乘法

组成—扩展无符号乘法器(增加S_P及相关电路)

控制流程—①结果符号计算

②循环的判断-加法-移位操作

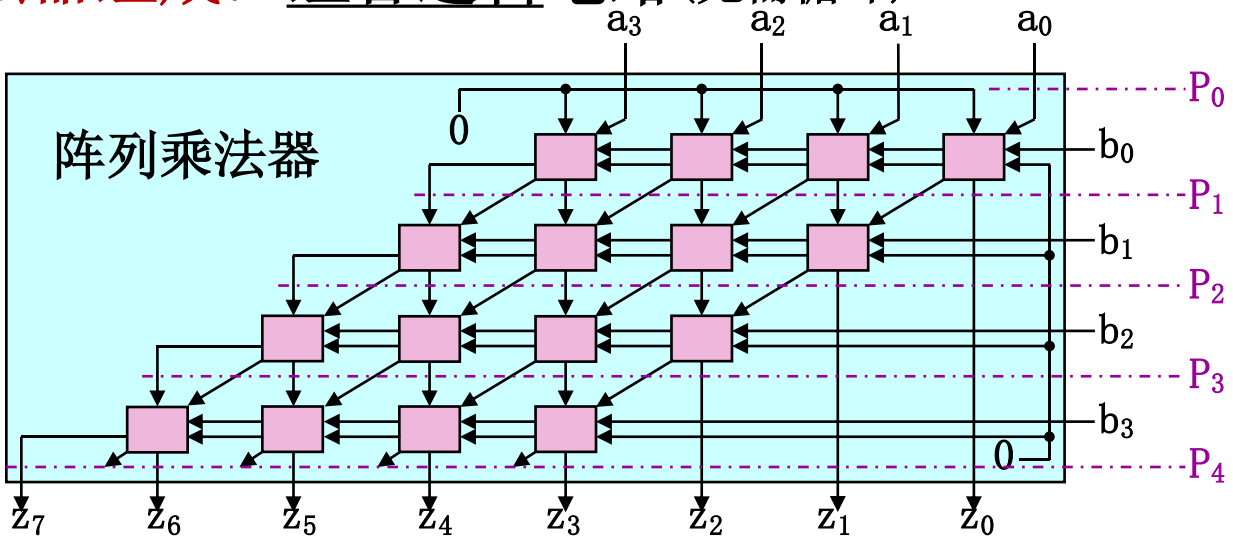
③拼接成乘积(符号+数值)

*溢出判断：小数乘法不会溢出(与乘积长度无关)

4、阵列乘法器

(快速乘法)

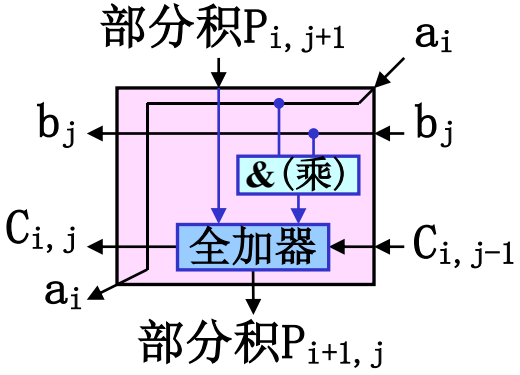
*阵列乘法器组成：组合逻辑电路(无需循环)



位乘元件组成—$P_{i+1,\,j}=(P_{i,\,j+1}+a_i\times b_j+C_{i,\,j-1})$

运算时延—$T=n\cdot T_{位乘}<n\cdot T_{CLK}$

(一位乘法器)



*阵列乘法器功能：

无符号乘法 (实现其他乘法需增加辅助电路)

§2.4 浮点数的运算

※主要内容：浮点加减的运算方法（不要求逻辑实现）

*浮点数的表示：IEEE 754标准，$F=(-1)^{S_M}\times M\times 2^E$

进制—M为二进制，E为二进制

格式—浮点格式(M为小数、E为整数)，2种长度(单精度、双精度)



编码—M为原码，E为移码(偏移值为$2^{Len(E)}-1$)

└←规格化数为1.M，非规格化数为0.M

规格化—目标是$0.5\leqslant|M|<1$；

操作类型有左规(M<<1位、E－1)、右规(M>>1位、E＋1)

1、浮点加减运算

***运算步骤：**

基本运算过程—设浮点数$A=M_A\times 2^{E_A}$、$B=M_B\times 2^{E_B}$，且$E_A\leqslant E_B$

①对阶—结果的阶$=E_B$，$A=(M_A\times 2^{E_A-E_B})\times 2^{E_B}$

②尾数加减—结果的尾数$=M_A\times 2^{E_A-E_B}\pm M_B$ ←有效位数受损

③结果规格化—使结果为规格化数 ←提高结果精度

减少精度损失的方法—

运算过程中，增加尾数的有效位数（增加部分称为附加位）

如：IEEE 754标准中，附加位至少含保护位、舍入位

运算步骤—

对阶、尾数加减、结果规格化、尾数舍入

（教材中“溢出判断”，可融入结果规格化、尾数舍入步骤中）

*运算方法：

设浮点数$A=M_A\times 2^{E_A}$、$B=M_B\times 2^{E_B}$，尾数加减后$F'=M'_F\times 2^{E'_F}$，

规格化后$F''=M''_F\times 2^{E''_F}$，尾数舍入后$F=M_F\times 2^{E_F}$ ←M'_F、M''_F含附加位

例1—浮点数的尾数用5位补码、阶用3位补码表示，附加位为2位，采用双符号位运算。$A=0.1101\times 2^{01}$、$B=-0.1010\times 2^{11}$，求$[A+B]_{浮}$

①对阶—(1)求公共阶：$E'_F=\max(E_A, E_B)$，$\Delta E=|E_A-E_B|$

(2)小阶尾数右移：右移ΔE位，移出的位→附加位

解-1：$[E'_F]_{补}=00\ 11$，$\Delta E=|00\ 01-00\ 11|=10$，

$[M'_A]_{补}=00.0011\ 01$，$[M'_B]_{补}=11.0110\ 00$

思考①：小数何时被大数吃掉？ $\Delta E\geq$尾数长度时

②尾数加减—$M'_F=M'_A\pm M'_B$，尾数及附加位一起运算

解-2：$[M'_F]_{补}=00.0011\ 01+11.0110\ 00=11.1001\ 01$

思考②：结果溢出怎么办？ 不视为错误，尾数规格化时再处理

③结果规格化—$M''_F=M'_F$规格化结果，$E''_F=E'_F+k$；

判断E''_F是否溢出

规格化：

$|M'_F|\geqslant 1.0$时—$M''_F=M'_F$右规1次($M'_F<2$)，　$k=1$

$0.5\leqslant|M'_F|<1.0$时—$M''_F=M'_F$(已为规格化数)，　$k=0$

$|M'_F|<0.5$时—$M''_F=M'_F$左规x次(直到$\geqslant 0.5$)，$k=-x$

思考：M'_F为负数、补码编码时，3种范围有何变化？　>，<及≤，≤

溢出判断：判断方法～阶的运算方法(如单/双符号、补/移码)，

溢出时进行相应处理

IEEE 754标准：E''_F上溢时，$F=\pm\infty$($[E_F]_{移}=255, [M_F]_{原}=0$)

E''_F下溢时，$F=0$($[E_F]_{移}=0, [M_F]_{原}=0$)

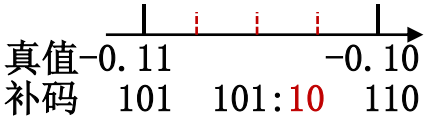
解-3：因$[E'_F]_{补}=00\ 11$，$[M'_F]_{补}=11.1001\ 01$，

故需左规1次，$[M''_F]_{补}=11.0010\ 10$，$[E''_F]_{补}=00\ 10$；

E''_F未溢出

④尾数舍入—$M_F = M''_F + y$，y根据附加位得到；

M_F溢出时进行规格化(含溢出判断)



舍入类型：截断法(朝0舍)，恒置法(朝∞入)，舍入(就近舍入)

(针对真值)　($y=0$)　($y=1$[正数]/-1[负数])　($y=$附首)

教材P76第17行那一段，说明的是基于机器数舍入的方法，更正如下：
由于负数机器数的舍使绝对值变大、入使绝对值变小，为了针对真值舍入，负数机器数采用如下规则：①所有位全为0时，不舍不入；②最高位为0、其余位不全为0时，或最高位为1、其余位全为0时，进行舍处理；③最高位为1、其余位不全为0时，进行入处理。就是说，附加位机器数在01/10时舍、11时入、00时弃。

IEEE 754标准：默认采用就近舍入法(4舍/6入/5成偶数)

即($M''_{F附}$=10…0)($M''_{F末}$=1)时$y=1$，否则$y=M''_{F附首}$

查表舍入法

解-4：因$[M''_F]_{补}$=11.0010 10，$[E''_F]_{补}$=00 10，

M''_F为负数，附加位10选择舍，

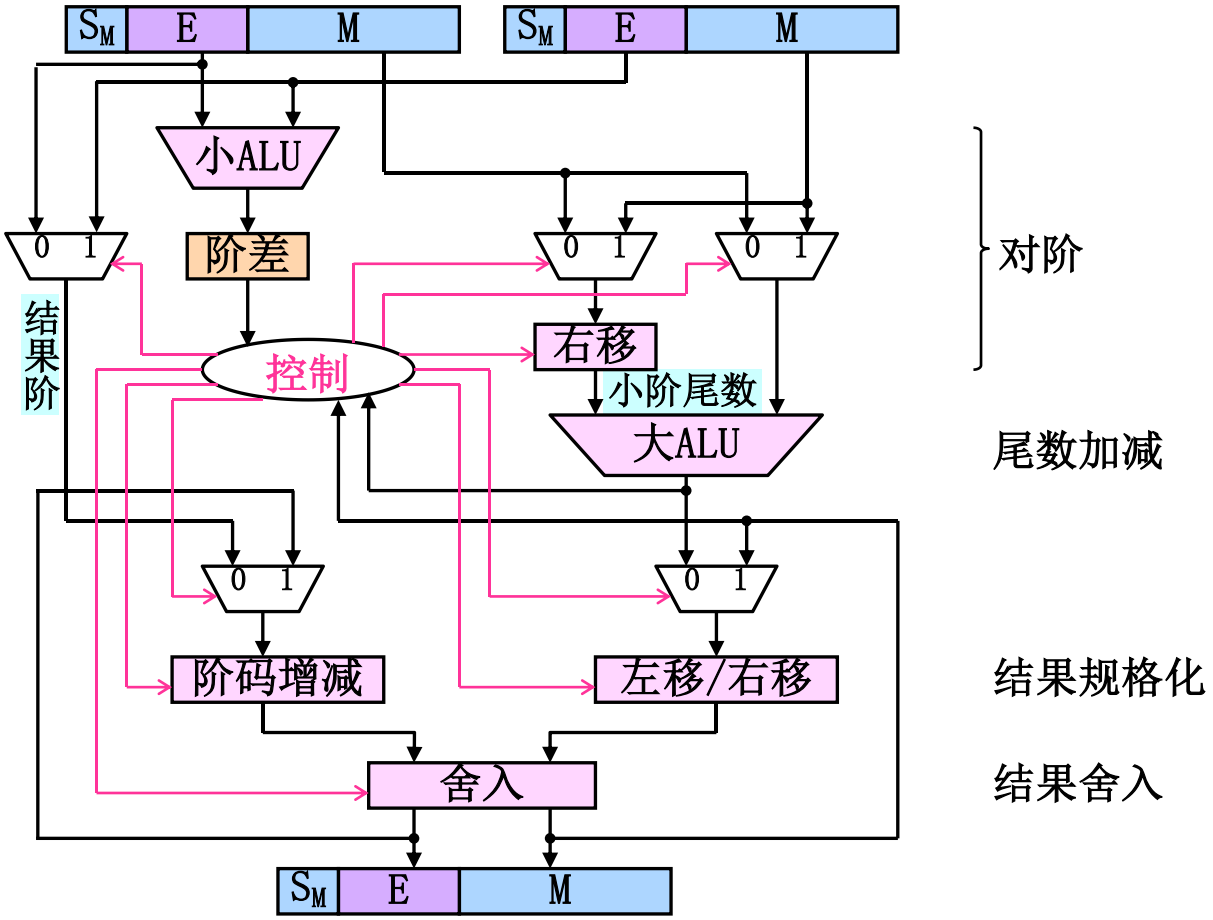
故$[M_F]_{补}$=1.0010，$[E_F]_{补}$=0 10

练习—浮点数的尾数用6位原码、阶用4位移码表示，附加位为2位，采用单符号位运算，尾数采用舍入法。求$0.11001\times2^{6}-0.10101\times2^{4}$的浮点数

*运算实现：

方法— 4个步骤按序进行

结构—



2、浮点乘除运算　　（不要求，了解）

*运算方法：　设浮点数$A=M_A\times 2^{E_A}$、$B=M_B\times 2^{E_B}$

①操作数预判—判断M_A及M_B是否符合要求(如＝0或∞)

②阶码运算—结果的阶＝E_A+E_B或E_A-E_B

③尾数运算—结果的尾数＝$M_A\times M_B$或$M_A\div M_B$

④结果规格化—使结果为规格化数　　←同浮点加减运算

⑤尾数舍入—附加位处理

注：步骤②、④及⑤会改变阶，需判断溢出

§2.6 运算器的组成

※主要内容：ALU的组成、运算器的组成(部件/互连)

一、常用数字逻辑电路

*组合逻辑电路：又称操作单元，输出＝f(输入)

类型—三态门、译码器、选择器、加法器等 (P48~)

	功能	输入信号线	输出信号线
三态门	输出三种状态	A、 ①?	Y
译码器	每个编码都有不同的电位	D_2~D_0、 ③?	②?
选择器	从多个输入中选择一个输出	D_7~D_0、 ④?	Y
加法器	进行全加运算	A、B、 ⑤?	F、 ⑥?

练习—利用上述器件及门电路，设计如下功能电路

(1) $Y=\begin{cases} A\cdot B & E=0\text{时} \\ B & E=1\text{时} \end{cases}$

(2) 供血型X_1X_0 → 血型配对 → 结果S（1-成功）

受血型Y_1Y_0 →

血型：00～11=O/A/B/AB

配对：O→O、O/A→A、O/B→B、4种→AB

*时序逻辑电路：又称状态单元，输出＝f(输入, 当前状态)

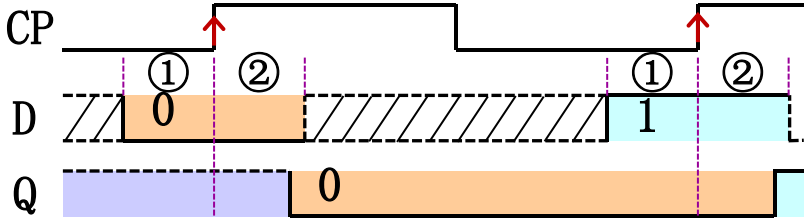
类型—触发器、锁存器、寄存器、计数器，及存储器

（边沿触发）（电平触发）（边沿触发）（边沿触发）

	操作功能	输入信号	输出信号
触发器	写入、保持、清零	D、CP、R	Q
锁存器	写入、保持、清零	D、①?、R	Q
寄存器	写入、保持、清零	D_7~D_0、②? R	③?
计数器	计数、保持、清零、④?	CP、⑤? R	Q_2~Q_0

思考：为什么没有读出操作？ 保持就是读出

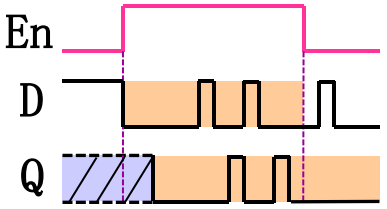
触发器—写入值D的时序应满足建立时间、保持时间的要求



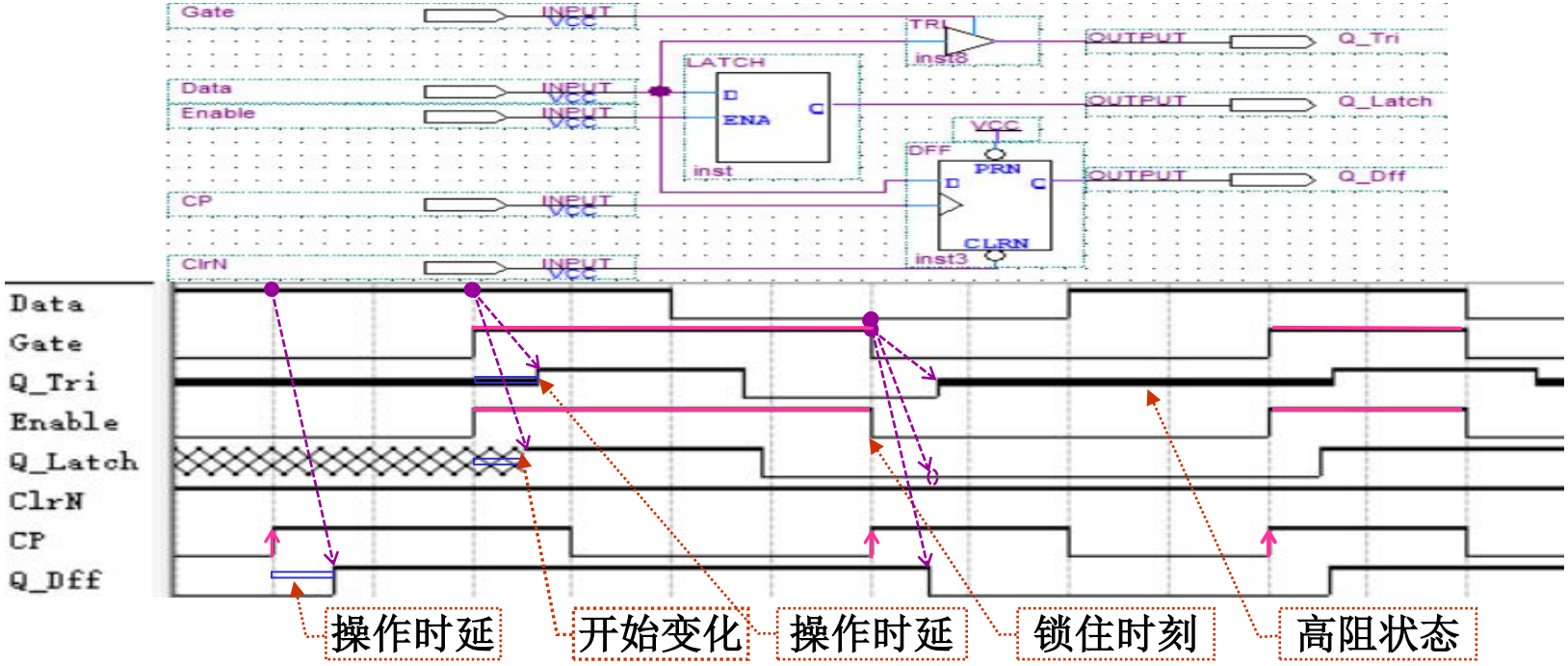
①建立时间setup

②保持时间Hold

锁存器—控制信号E应在数据稳定时无效

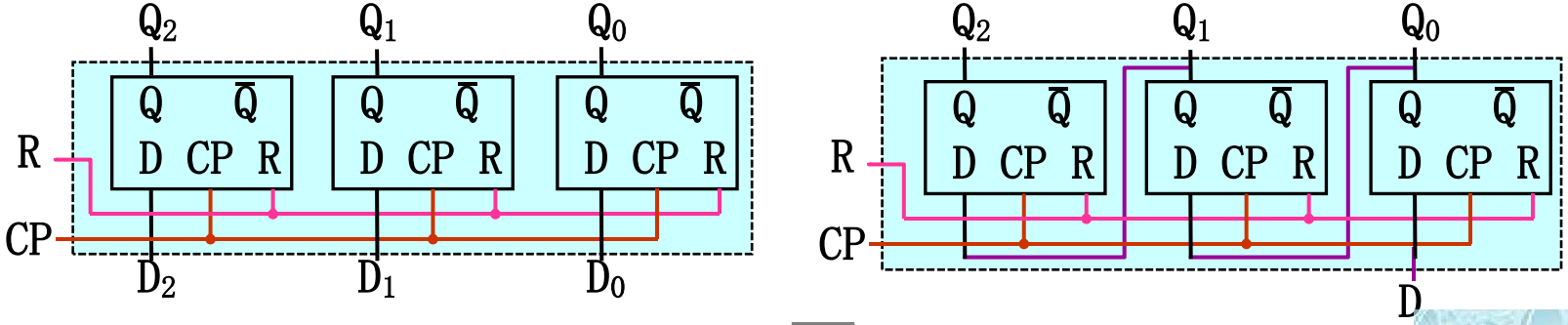


示例—三态门、触发器、锁存器的特性(Quartus II的时序仿真)



寄　存　器—由多个触发器并联构成

移位寄存器—功能有串→并、并→串、寄存器等

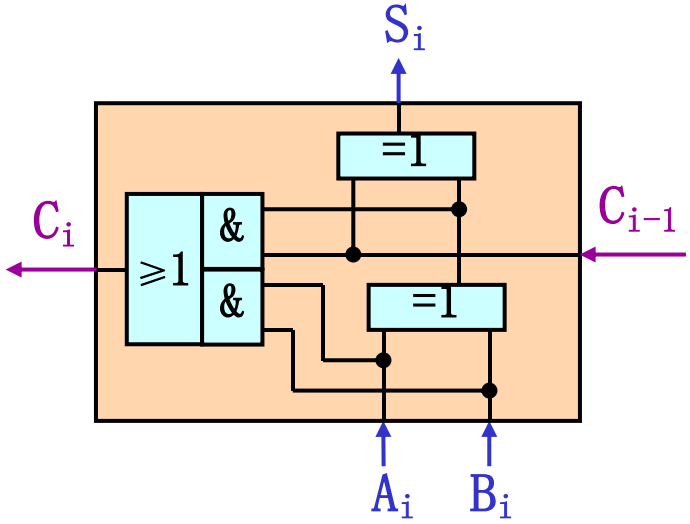


二、加法器的组成

1、全加器

*真值表：半加器没有C_{i-1}

	A	B	C_{i-1}	进位C_i	全加和S_i
真值表	0	0	0	0	0
	0	1	0	0	1
	1	0	0	0	1
	1	1	0	1	0
	0	0	1	0	1
	0	1	1	1	0
	1	0	1	1	0
	1	1	1	1	1



*内部逻辑：

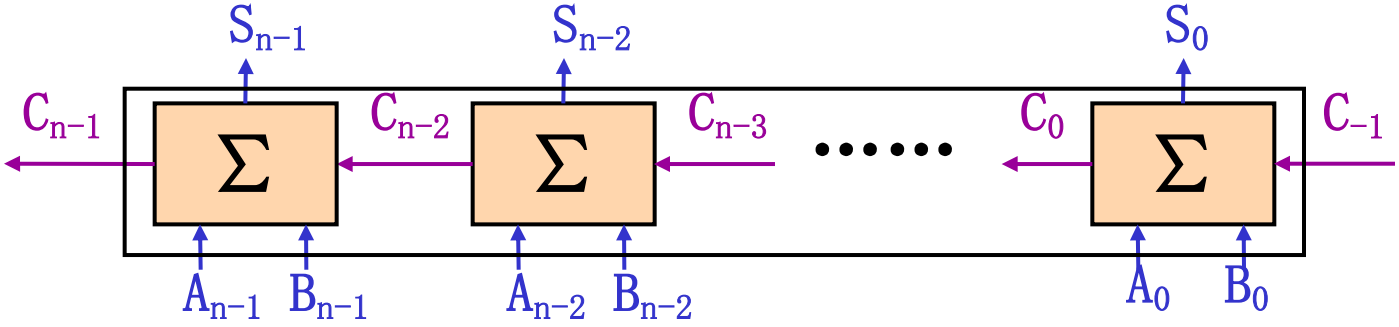
$$S_i = \overline{A_i}B_i\overline{C_{i-1}} + A_i\overline{B_i}\,\overline{C_{i-1}} + \overline{A_i}\,\overline{B_i}C_{i-1} + A_iB_iC_{i-1}$$

$$= \overline{A_i}(B_i\overline{C_{i-1}} + \overline{B_i}C_{i-1}) + A_i(\overline{B_i}\,\overline{C_{i-1}} + B_iC_{i-1}) = A_i \oplus B_i \oplus C_{i-1}$$

$$C_i = A_iB_i\overline{C_{i-1}} + \overline{A_i}B_iC_{i-1} + A_i\overline{B_i}C_{i-1} + A_iB_iC_{i-1} = A_iB_i + (A_i \oplus B_i)C_{i-1}$$

2、串行进位加法器

*组成：n个1位全加器＋串行进位逻辑



串行进位逻辑— 串行形成$C_0 \rightarrow C_{n-1}$，$C_i = A_iB_i + (A_i \oplus B_i)C_{i-1}$

*特点：进位电路简单，运算速度慢($T_{加法} = n \cdot \Delta t$)

3、先行进位加法器

目标—同时形成$C_{n-1} \sim C_0$，以提高运算速度($\approx \Delta t$)

*进位辅助函数：假设$C_i = A_iB_i + (A_i \oplus B_i)C_{i-1} = G_i + P_iC_{i-1}$

产生函数G_i— $G_i = A_iB_i$，本地产生进位(不依赖于C_{i-1})

传递函数P_i— $P_i = A_i \oplus B_i$，本地传递进位(依赖于C_{i-1})

*4位先行进位电路：

逻辑— C_0～C_3同时形成，其中$G_i=A_iB_i$，$P_i=A_i\oplus B_i$

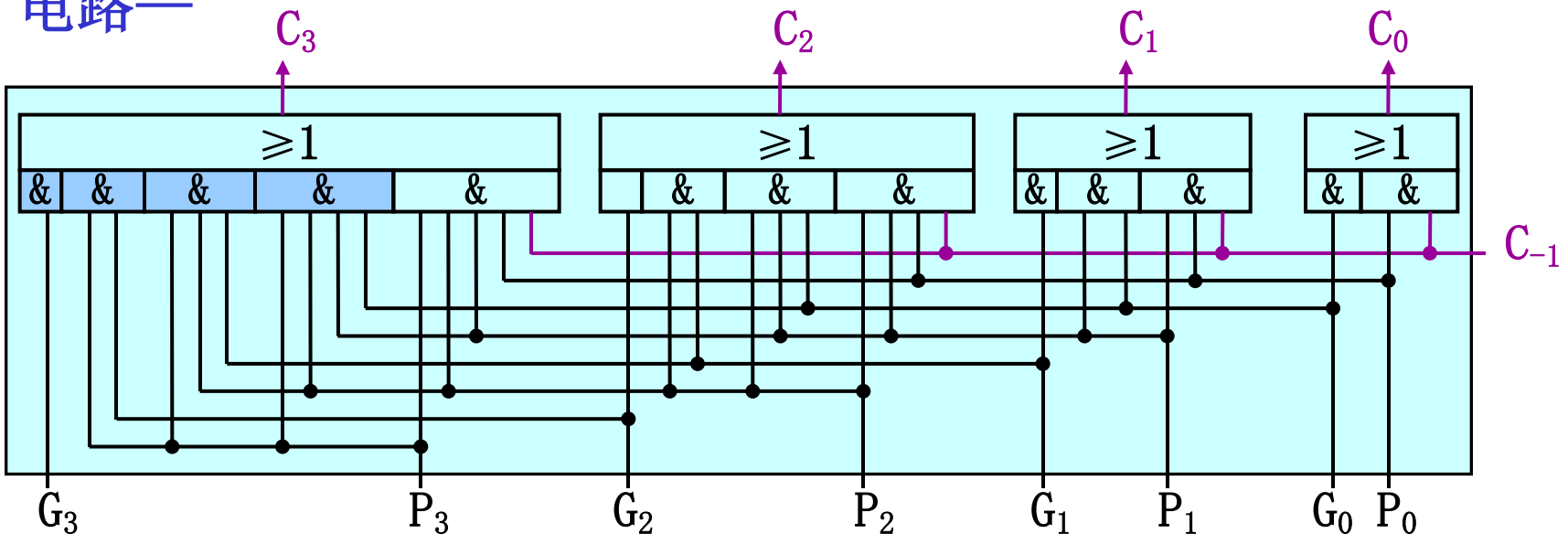
$C_0=G_0+P_0C_{-1}$

$C_1=G_1+P_1C_0=G_1+P_1G_0+P_1P_0C_{-1}$

$C_2=G_2+P_2C_1=G_2+P_2G_1+P_2P_1G_0+P_2P_1P_0C_{-1}$

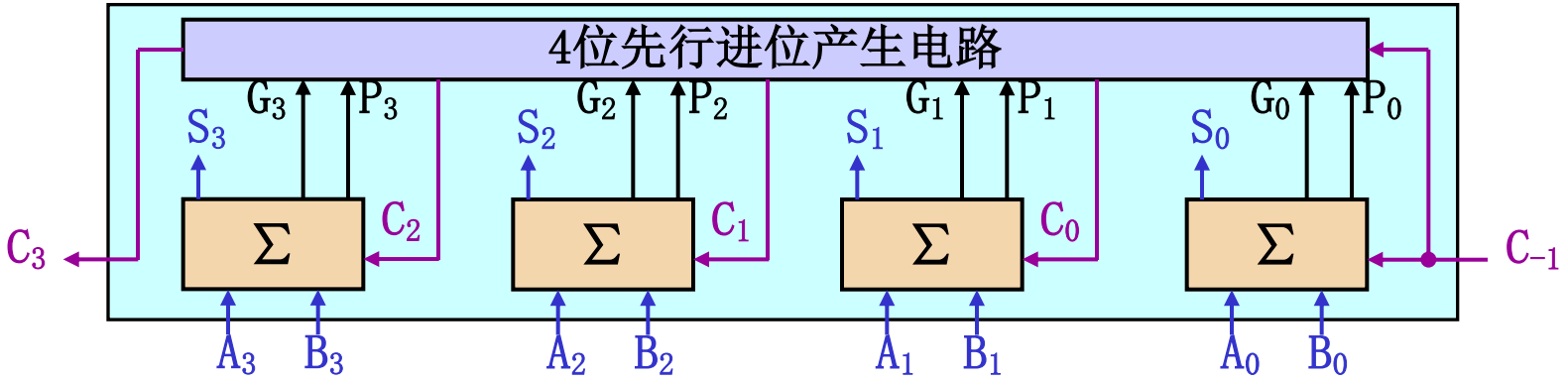
$C_3=G_3+P_3C_2=G_3+P_3G_2+P_3P_2G_1+P_3P_2P_1G_0+P_3P_2P_1P_0C_{-1}$

电路—

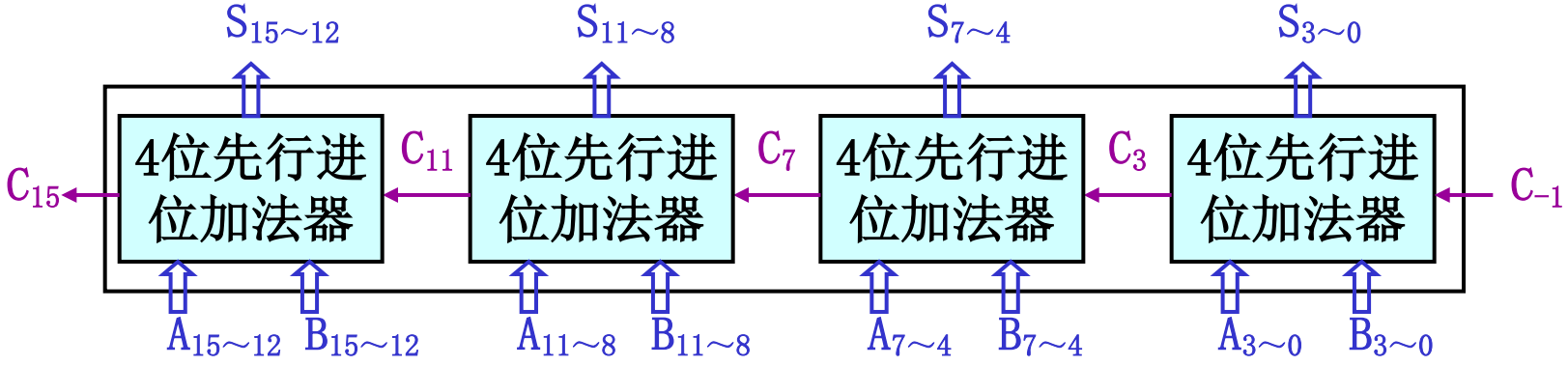


*n位先行进位加法器的组成：

4位先行进位加法器—全加器需输出G、P



n位先行进位加法器—有全并行、组间串行、组间并行3种（下图）



三、运算器的组成及结构

1、运算器的功能与组成

***运算器的功能：** 实现ISA中各指令的约定功能　←组成是逻辑实现

指令功能示例— 子功能包括取操作数、运算、写结果(含标志)

助记符	指令功能	标志ZF/CF/OF/SF的产生	说明
ADD/SUB	Rd←Rd±Rs	运算产生	Rs、Rd表示寄存器； M[a]表示a#存储单元； Imm表示立即数； 顺序型指令的下条指令地址为PC←PC+1
XOR	Rd←Rd⊕M[Rs]	CF/OF＝0，其余=运算	
SHL	Rd←Rd<<Imm	运算产生	
MUL	(Rs,Rd)←Rd*Rs	CF/OF＝运算，其余任意	
JNZ	ZF=0时PC←Rs 否则PC←PC+1	不改变	

指令功能特点— 操作数常放在REG、MEM及指令字(立即数)中；

不同指令的运算功能不同时使用；　←指令串行执行

不同指令产生的标志不同；　←操作特性不同

指令产生的标志需保存使用　←供后续指令用

如JNZ需使用ZF→┘

实例：R0←R0－R1

*运算器的部件组成：　—基于ISA需求

运算部件—有多种(运算方法不同)，每种含多种功能(复用元器件)

		表示格式	运算方法	运算类型	运算部件
数值数据	整数	定点(无符号)	算术	加/减、移位	ALU、移位器
		定点(有符号)		乘/除等	乘除法器等
	实数	浮点	浮点	加/减/乘/除等	FPU
非数值数据	逻辑数	位向量	逻辑(按位)	与/或/非等	ALU
	字符	定点(无符号)	算术＋逻辑	＞/≤/≠等	ALU＋门电路
	其他	定点/向量	饱和	饱和加减等	MMX或SSE

算术运算基于逻辑运算实现

存放部件— 寄存器组(存放数据/地址)、　←按地址访问(读/写)

状态寄存器PSR(存放标志等)　←可能写入部分标志

*运算器的部件互连：　—基于ISA需求

部件引脚连接—汇总各指令的需求，增设连接电路

部件互连结构—有总线结构、专用结构2类　(稍后讨论)

PSR—Program Status Register

2、运算器的部件设计

*设计步骤：组织各部件功能(分类合并所有运算&操作需求)，　←复用器件

设计各部件电路

思考：组合逻辑电路设计的步骤？

①逻辑抽象(真值表) ②逻辑化简(表达式)

③电路设计(内部逻辑)

*ALU设计示例：

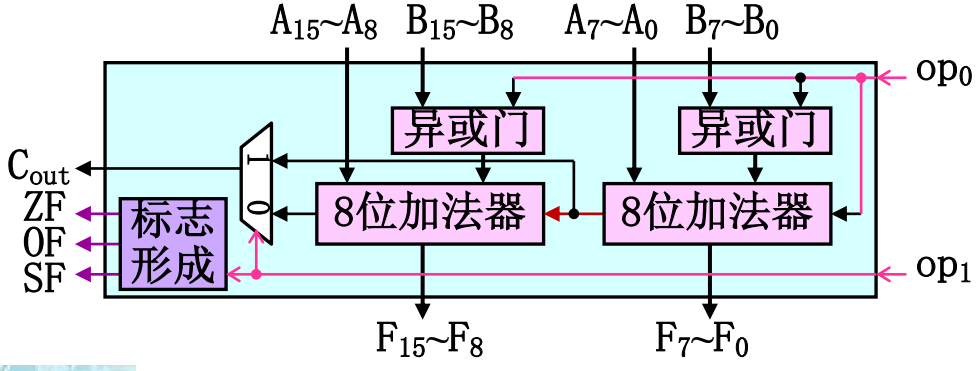
例1：设计支持8位及16位加法及减法的ALU，产生C_{out}及ZF/OF/SF

功能表—输入为$A_{15\sim0}$及$B_{15\sim0}$、op_{1-0}，输出为$F_{15\sim0}$、C_{out}及3个标志

逻辑函数—

op_1op_0	00	01	10	11
$F_{15}\sim F_0$	$A_{15\sim0}+B_{15\sim0}$	$A_{15\sim0}-B_{15\sim0}$	$F_{7\sim0}=A_{7\sim0}+B_{7\sim0}$	$F_{7\sim0}=A_{7\sim0}-B_{7\sim0}$
C_{out}	C_{15}		C_7	
ZF/OF/SF	按16位结果形成		按8位结果形成	

内部逻辑—2个8位加减法器级联，用op_1控制C_{out}及形成标志



标志形成逻辑：

$ZF=op_1?\ \sim(F_7+\cdots+F_0):\sim(F_{15}+\cdots+F_0)$

$OF=op_1?(C_7\oplus C_6):(C_{15}\oplus C_{14})$

$SF=op_1?\ F_7:F_{15}$

例2：设计支持A&B、A⊕B的8位加法器；基于所设计加法器，设计支持A+B、A−B、A+1、A−1、A&B、A⊕B的8位ALU，产生C_{out}及ZF/OF/SF

①加法器输出函数—

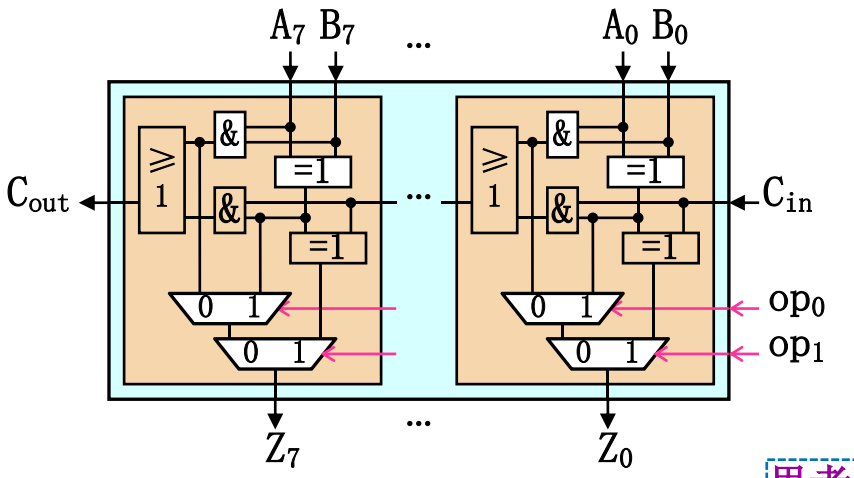
$op_{1\text{-}0}$	00	01	1×
Z_7~Z_0	A&B	A⊕B	A+B

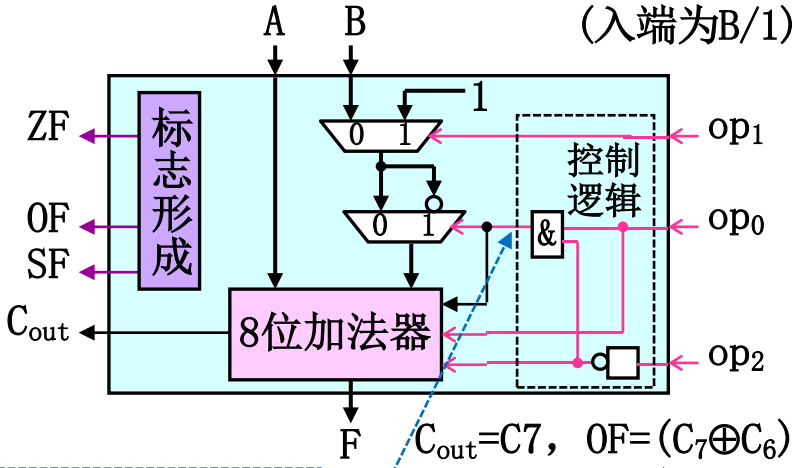
加法器内部逻辑—逻辑运算复用加法运算结果

②ALU输出函数—

$op_{2\text{~}0}$	000	001	010	011	100	101
F_{15}~F_0	A+B	A−B	A+1	A−1	A&B	A⊕B
C_{out}	加法器最高位进位					
ZF/OF/SF	按运算结果形成					

ALU内部逻辑—逻辑运算复用加法器，加减运算增加数据源

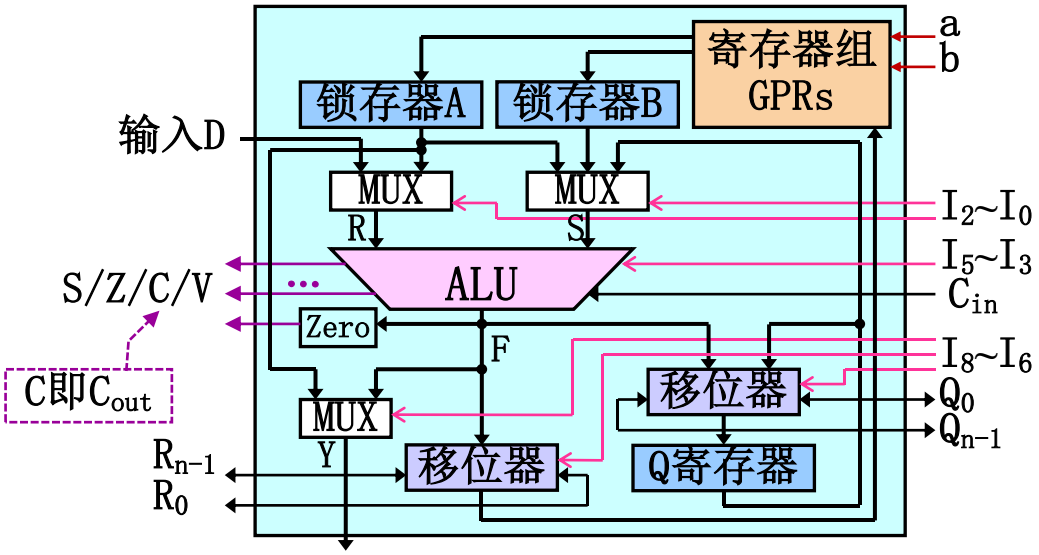




思考：如何避免A&~B?

C_{out}=C7，OF=(C_7⊕C_6)，SF=F_7，ZF=~(F_7+…+F_0)

例3：分析运算器29C101的移位功能，写出实现指令Rb←Rb-Ra、Rb←(Ra+Rb)>>$_L$1功能的控制信号，说明其如何支持乘法运算



不写寄存器

	$I_{8\sim6}$			$I_{5\sim3}$	$I_{2\sim0}$	
	GPR[b]	Q	Y	ALU	R	S
000		F	F	R+S	A	Q
001			F	S-R	A	B
010	F		A	R-S	0	Q
011	F		F	R∨S	0	B
100	F/2	Q/2	F	R∧S	0	A
101	F/2		F	$\overline{R}$∧S	D	A
110	2F	2Q	F	R⊕S	D	Q
111	2F		F	R⊙S	D	0

移位功能—左移1位(如F<<1[移入R_0])、右移1位(如Q>>1[移入Q_{n-1}])、直通，2个移位器可级联(Q_{n-1}=R_0)

控制信号—指令Rb←Rb-Ra： 011 001 001

思考：改为>>$_A$1呢？

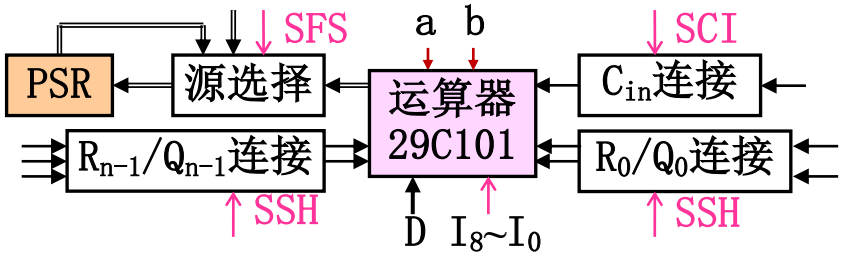
指令Rb←(Rb+Ra)>>$_L$1： 101 000 001、R_{n-1}=0

乘法支持—每次乘1位，外部循环实现n位乘法

有/无符号乘法

└←Q+A或0，GPR[b]←F/2、Q←Q/2，R_{n-1}=C或S、Q_{n-1}=R_0

3、运算器的部件互连



***部件的引脚连接：**（以29C101为例）

连接需求示例—基于指令功能

助记符	指令功能例	指令保存的标志	C_{in}	R_{n-1}	Q_{n-1}	R_0	Q_0	ZF	CF	OF	SF
加法	Rd←Rd＋Rs	运算产生	0	*	*	*	*	Z	C	V	S
减法	Rd←Rd－Rs		1	*	*	*	*		C#		
带进位加	Rd←Rd＋Rs＋CF		CF	*	*	*	*		C		
带借位减	Rd←Rd－Rs－CF		CF#	*	*	*	*		C#		
异或	Rd←Rd⊕Rs	CF/OF=0，其余产生	*	*	*	*	*	Z	0	0	S
无符号乘	(Rs,Rd)←Rd*Rs	产生OF，其余任意	0	C	R_0	/	/	*	*	Vu	*
有符号乘	(Rs,Rd)←Rd*Rs		0	S	R_0	/	/	*	*	Vs	*
无符号除	(Rs,Rd)←Rd/Rs	均任意	0	/	/	Q_{n-1}	C	*	*	*	*

连接电路示例— 用电路实现数据源，用选择器连接引脚

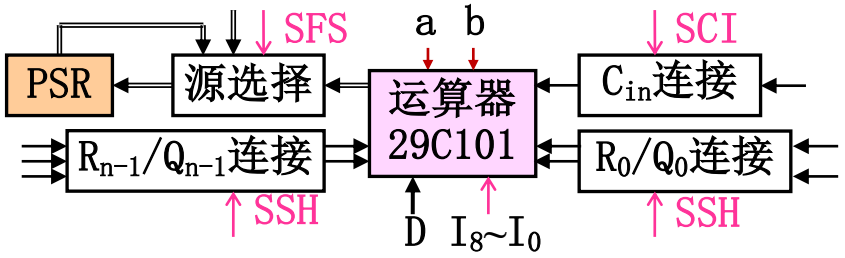
思考：为何左移及右移连接的控制信号可相同？

SCI	C_{in}
00	0
01	1
10	CF
11	CF#

SSH	左移		右移	
	R_0	Q_0	R_{n-1}	Q_{n-1}
0	0	*	C	R_0
1	Q_{n-1}	C	S	R_0

SFS	ZF	CF	OF	SF
000	Z	C	V	S
001	Z	C#	V	S
010	Z	0	0	S
100	*	*	Vu	*
101	*	*	Vs	*

例4： 29C101引脚连接的控制逻辑如下(同上页)，写出实现表中指令功能的控制信号



	$I_{8\sim6}$			$I_{5\sim3}$	$I_{2\sim0}$	
	GPR[b]	Q	Y	ALU	R	S
000		F	F	R+S	A	Q
001			F	S-R	A	B
010	F		A	R-S	0	Q
011	F		F	R∨S	0	B
100	F/2	Q/2	F	R∧S	0	A
101	F/2		F	R∧S	D	A
110	2F	2Q	F	R⊕S	D	Q
111	2F		F	R⊙S	D	0

SCI	C_{in}
00	0
01	1
10	CF
11	CF#

SSH	左移		右移	
	R_0	Q_0	R_{n-1}	Q_{n-1}
0	0	*	C	R_0
1	Q_{n-1}	C	S	R_0

SFS	ZF	CF	OF	SF
000	Z	C	V	S
001	Z	C#	V	S
010	Z	0	0	S
100	*	*	Vu	*
101	*	*	Vs	*

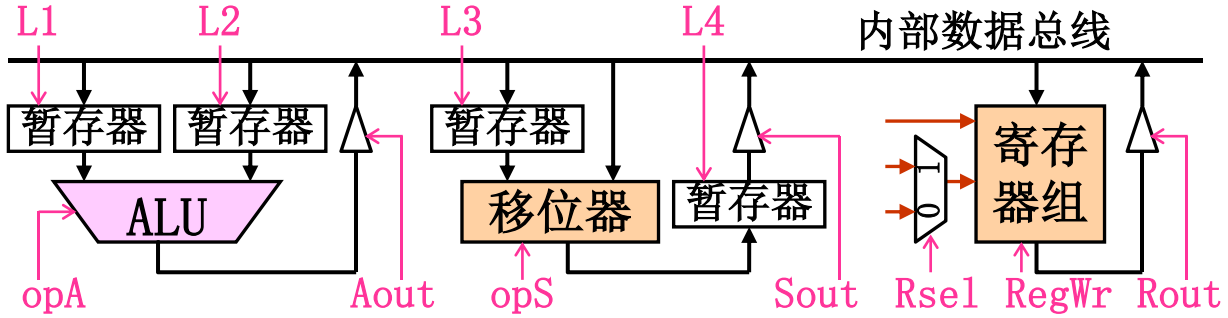
指令 / 控制信号	b	a	SFS	I_8~I_6	I_5~I_3	I_2~I_0	SCI	SSH
R2←R2-R1，PSR←Z/C/V/S	0010	0001	001	011	001	001	01	/
R2←R2⊕R1，PSR←Z/0/0/S	0010	0001	010	011	110	001	00	/

*部件的互连结构：

锁存器或触发器

总线结构—部件出端设置三态门，部件入/出端设置暂存器

（避免总线上信号冲突）　　（避免端口间信号干扰）



opA—
00/01/10/11
为+/-/&/⊕

opS—
00/10/11为
<< / >>L / >>A

思考①：组合逻辑电路最多有几个端口的信号不暂存？ 1个

思考②：为何寄存器组的端口信号都不暂存？ 仅需处理同时使用的端口信号

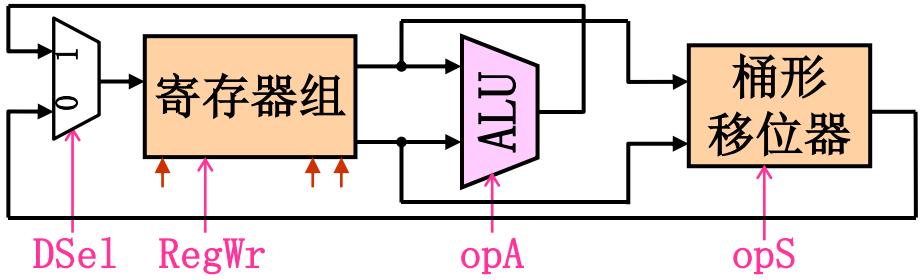
每步结果需保存在暂存器中

例5：基于上述互连通路，给出实现下列指令功能的控制信号

指令功能	实现步骤	L1	L2	opA	Aout	L3	opS	L4	Sout	Rsel	RegWr	Rout
R2←R0⊕R1	(1)取R0→暂存器	1	*	*	0	*	*	*	0	0	0	1
	(2)取R1→暂存器	0	1	*	0	*	*	*	0	1	0	1
	(3)运算、结果→R2	0	0	11	1	*	*	*	0	*	1	0
R5←R3>>LR4	(1)取R3→暂存器	*	*	*	0	1	*	*	0	0	0	1
	(2)取R4、运算	*	*	*	0	0	10	1	0	1	0	1
	(3)结果→R5	*	*	*	0	*	*	0	1	*	1	0

专用结构—部件入端设置选择器

←性能好、控制复杂



opA—00/01/10/11
为+/-/&/⊕
opS—00/10/11
为<< / >>$_L$ / >>$_A$

例6：基于上述互连通路，给出实现表中指令功能的控制信号

指令功能	DSel	RegWr	opA	opS	说明
R2←R0+R1	1	1	00	**	
R2←R0⊕R1	1	1	11	**	
R5←R3<<R4	0	1	**	00	
R5←R3>>$_L$R4	0	1	**	10	

影响指令性能的因素—

部件设计影响操作时延(如1位乘/快速乘)，

互连结构影响实现过程/性能(步骤)

作业2-4：P87起—30(1)、31(1)、34(1)、36

※附1：模2乘法、除法

模2乘法—部分积求和采用模2加法(无进位)求得

模2除法—位商为部分余数的首位，

余数采用模2减法(即模2加法)求得

```
      1 0 1 0
×       1 0 1
---------------
      1 0 1 0
    0 0 0 0
  1 0 1 0
---------------
  1 0 0 0 1 0
```

```
            1 0 1      ←商无意义
        ------------
1 0 1 ) 1 0 0 0 0
        1 0 1
        ---------
          0 1 0
          0 0 0
          -----
            1 0 0
            1 0 1
            -----
              0 1      ……余数
```